

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ
БЕЛАРУСЬ
Учреждение образования «БЕЛОРУССКИЙ
ГОСУДАРСТВЕННЫЙ
ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ»

Факультет Информационных технологий
Кафедра Программной инженерии
Специальность 6-05-0612-01 «Программная инженерия»

**ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
КУРСОВОГО ПРОЕКТА:**

по дисциплине «Объектно-ориентированные технологии»
Тема: Программное средство «Автосервис»

Исполнитель
Студент 2 курса группы 7 Пинчук Николай Александрович
(Ф.И.О.)

Руководитель работы преп.-стажёр Авдеева В.Д.
(учен. степень, звание, должность, подпись, Ф.И.О.)

Курсовой проект защищен с оценкой

Председатель Белодед Н.И.
(подпись)

Оглавление

Введение	6
1 Аналитический обзор прототипов и литературных источников	7
1.1 Обзор аналогичных решений	7
1.2 Постановка задачи	10
2 Анализ требований к программному средству и разработка функциональных требований	13
2.1 Описание средств разработки	13
2.2 Спецификация функциональных требований	14
2.3 Диаграмма вариантов использования.....	15
3 Проектирование программного средства.....	16
3.1 Проектирование архитектуры приложения	17
3.2 Проектирование структуры базы данных	19
3.3 Структура проекта.....	21
3.4 Диаграмма классов	22
4 Реализация программного средства.....	22
4.1 Гибкость системы.....	23
4.1.1 Классы для реализации паттерна MVVM.....	23
4.1.2 Классы для работы с базой данных	25
4.2 Реализация функционала приложения	26
4.2.1 Метод авторизации пользователя в приложении.....	26
4.2.2 Метод регистрации клиента в приложении.....	26
4.2.3 Методы клиента: просмотр услуг и цен, запись на ремонт	27
4.2.4 Отслеживание статуса заказа и обратная связь клиента	27
4.2.5 Методы администратора: расписание, заказы, склад, уведомления о смене статуса	27
4.2.6 Методы механика: задания на день, выполнение заказа, поиск, запрос запчастей	28
5 Тестирование, проверка работоспособности приложения и анализ результатов.....	29
5.1 Тестирование функций окна регистрации	29
5.2 Тестирование функций окна авторизации	30
5.3 Тестирование функций администратора.....	31

6 Руководство по использованию	33
6.1 Регистрация и авторизация.....	33
6.2 Использование приложения в роли клиента.....	34
6.3 Использование приложения в роли администратора.....	36
6.4 Использование приложения в роли механика	39
6.5 Заключение.....	40
Заключение.....	41
Список использованных источников.....	42
ПРИЛОЖЕНИЕ А	
ПРИЛОЖЕНИЕ Б	
ПРИЛОЖЕНИЕ Г	
ПРИЛОЖЕНИЕ Д.....	
ПРИЛОЖЕНИЕ Ж.....	
ПРИЛОЖЕНИЕ З	
ПРИЛОЖЕНИЕ И	

Введение

Тема разработки приложения «Автосервис» является актуальной и востребованной в современном мире, поскольку с ростом числа автомобилистов и увеличением спроса на качественный ремонт эффективное управление процессом технического обслуживания становится важной задачей как для автолюбителей, так и для сотрудников сервисных центров. Анализ существующих конкурентных решений показывает, что многие из них обладают как сильными сторонами, так и заметными недостатками, а в условиях растущей конкуренции и высоких ожиданий клиентов возникает потребность в новом, более удобном инструменте, объединяющем всех участников производственного процесса.

Целью данного курсового проекта является создание приложения, обеспечивающего прозрачное и удобное взаимодействие между клиентами, администраторами и механиками автосервиса. Для клиентов реализованы возможности регистрации и авторизации в системе, просмотра списка услуг с актуальными ценами, самостоятельной записи на ремонт, а также отслеживания текущего статуса заказа в режиме реального времени. Администратор, в свою очередь, получает инструменты для управления расписанием, полного цикла работы с заказами 0150 от создания до закрытия, контроля наличия запчастей на складе и отправки клиентам уведомлений при смене статуса заказа. Механики могут просматривать персональные задания на текущий день, отмечать выполнение работ, быстро находить нужные позиции через поиск по названию, а при необходимости запрашивать дополнительные запчасти в процессе ремонта.

При разработке приложения заложены ключевые принципы, направленные на достижение поставленных целей: интерфейс остаётся простым и понятным, что позволяет одинаково комфортно работать как опытным сотрудникам, так и обычным пользователям, процесс записи и оформления заказа построен с минимальными усилиями со стороны клиента, а эффективные инструменты коммуникации и уведомлений обеспечивают прозрачность взаимодействия между всеми ролями. В результате внедрения приложения пользователи получают возможность быстро находить нужные услуги, записываться на ремонт и контролировать выполнение работ, что в конечном итоге будет способствовать повышению удовлетворённости клиентов и улучшению качества обслуживания в автосервисе.

1 Аналитический обзор прототипов и литературных источников

1.1 Обзор аналогичных решений

Перед тем как приступить к разработке собственного программного приложения, важно провести тщательный анализ существующих аналогов. Этот процесс позволяет глубже понять потребности и предпочтения целевой аудитории, а также выявить ключевые тренды и недостатки на рынке. Исследование аналогичных решений помогает определить, какие функции и особенности наиболее востребованы пользователями, а также что можно улучшить или предложить в новой интерпретации. Кроме того, анализ конкурентов позволяет оценить их стратегии, методы привлечения клиентов и уровень обслуживания, что дает возможность разработать уникальное предложение, способное выделиться на фоне других. Такой подход не только повысит шансы на успех будущего приложения, но и обеспечит более точное соответствие ожиданиям пользователей.

В качестве первого аналога будет рассмотрен сайт «fitauto.ru» [1]. На рисунке 1.1 представлена главная страница сайта.

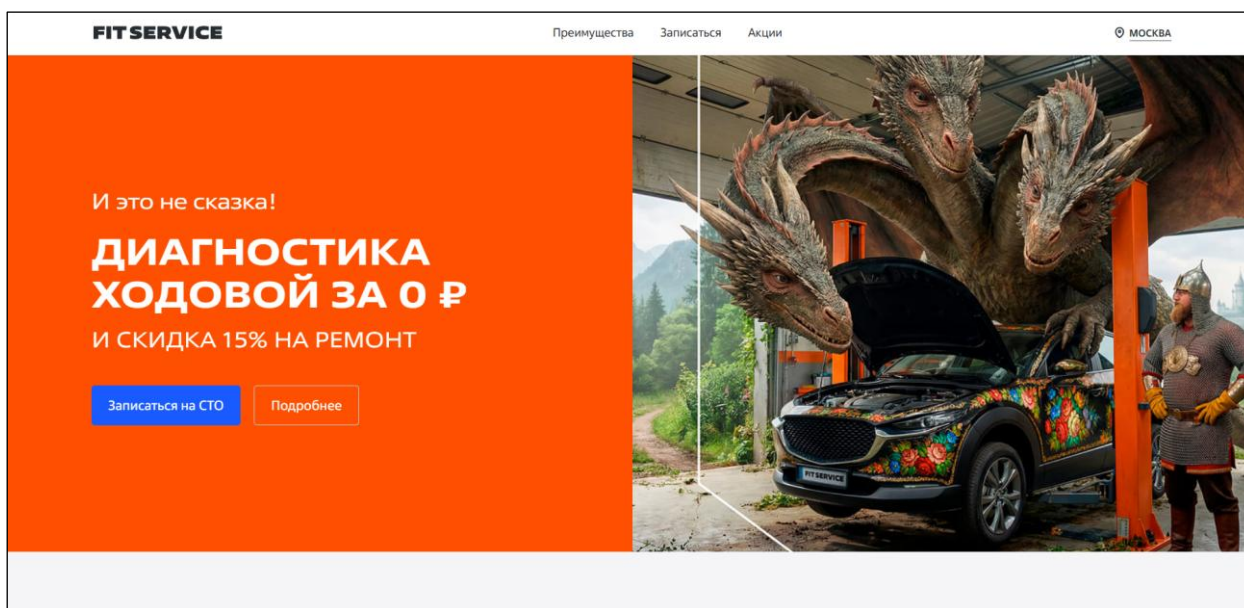


Рисунок 1.1 – Главная страница сайта «fitauto.ru»

Первым рассмотренным аналогом является сайт сети автосервисов FIT SERVICE, доступный по адресу fitauto.ru. Его главная страница выполнена в корпоративной цветовой гамме, где спокойный глубокий синий (#1E3A6F) сочетается с белым фоном и энергичным оранжевым (#F5821F), которым выделены ключевые кнопки призыва к действию – «Записаться на СТО» и «Подробнее». Такое цветовое акцентирование помогает пользователю сразу понять, какие действия являются основными.

Сайт предлагает форму предварительной записи, расположенную на видном месте; с её помощью можно выбрать станцию обслуживания, дату и время визита, а также подключиться к бонусной программе прямо после

оформления записи. При этом пользователю необходимо согласиться с политикой конфиденциальности и дать разрешение на получение информационных рассылок – типичный для современных сервисов механизм. В нижней части каждой страницы размещены контактные данные (телефон, ссылки на социальные сети) и иконки для скачивания мобильного приложения в Google Play, App Store и App Gallery, что свидетельствует о стремлении компании расширить каналы взаимодействия с клиентами.

Однако на сайте отсутствует возможность создания личного кабинета и авторизации, поэтому клиент не может отследить историю своих обращений, статус выполненных работ или повторить запись без повторного ввода данных. Верхнее меню навигации лаконично и включает разделы «Преимущества», «Записаться», «Акции», но главная страница при этом перегружена информационными блоками: крупный баннер с акцией, перечень услуг, карта расположения СТО, блоки с преимуществами. Несмотря на использование модульной сетки, такое обилие элементов создаёт ощущение визуальной перегруженности, что может затруднять быстрый поиск нужной информации. Страница с преимуществами содержит подробный перечень выгод (бесплатная диагностика, скидка 15% на ремонт, прозрачность работы, гарантия до года), но эти блоки статичны и не интерактивны. В целом сайт производит впечатление добротного корпоративного ресурса, однако отсутствие личного кабинета и недостаточная структурированность главной страницы являются существенными минусами с точки зрения удобства пользователя.

Обобщая все вышесказанное по данному сайту, можно сказать, что данный аналог является достаточно неплохим решением с интересными идеями. Однако, как и у любого другого сайта у него имеются свои недостатки, которые нужно учесть при проектировании собственного приложения.

Второй аналог – сайт willgood.ru – ориентирован на поиск шин и запись на сервисное обслуживание. Его дизайн выдержан в энергичной цветовой гамме, где яркий оранжевый (#FF6F00) и насыщенный чёрный (#1A1A1A) контрастируют с белым фоном. Оранжевый доминирует в кнопках, иконках и акцентных элементах, придавая сайту динамичность, а чёрный используется для текстов и футера, что создаёт чёткое разделение смысловых зон. Общая стилистика минималистична: крупные изображения товаров, простые формы и отсутствие лишних декоративных элементов. Функционально сайт предлагает стандартную строку поиска, позволяющую быстро находить нужные позиции, и содержит раздел с контактной информацией (адрес, телефон, электронная почта), традиционно вынесенный в футер.

На рисунке 1.2 представлена главная страница сайта.

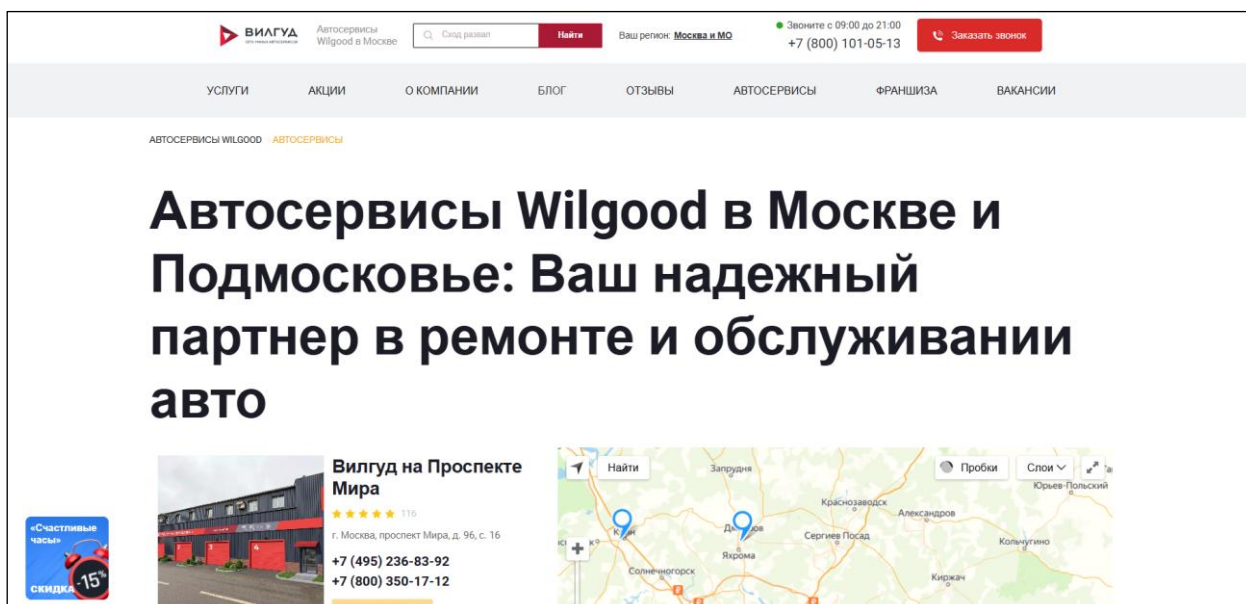


Рисунок 1.2 – Главная страница сайта «willgood.ru»

Однако возможности авторизации и личного кабинета здесь не предусмотрены, равно как и функционал онлайн-записи на сервис – пользователь может лишь найти контакты, но не оформить заявку непосредственно на сайте. Навигация построена интуитивно: основные разделы вынесены в верхнее меню, подменю раскрываются плавно, без резких переходов. Типографика заслуживает отдельного внимания: используются современные рубленые шрифты, крупные жирные заголовки легко читаются с экрана, межстрочные интервалы подобраны грамотно. Главная страница не перегружена, акцент сделан на визуальный контент и поисковую форму. Тем не менее отсутствие личного кабинета, возможности отслеживания заказов и обратной связи через личный кабинет снижают интерактивность ресурса. Сайт хорошо подходит для одноразового ознакомления с услугами, но не даёт клиенту инструментов для долгосрочного взаимодействия, что особенно важно для автосервисной тематики, где клиенты часто возвращаются повторно.

Третий проанализированный ресурс – arserv.by – представляет собой интернет-площадку, совмещающую каталог автозапчастей и услуги по ремонту автомобилей. В отличие от анализируемых ранее сайтов, arserv.by предлагает не просто продажу компонентов, а комплексное решение «запчасть + работа», что сокращает время поиска исполнителя. Пользователь может сравнить цены на оригинальные детали и их качественные аналоги, а также сразу увидеть стоимость типовых работ (замена масла, ремонт ходовой, диагностика). Тем не менее, отзывы о конкретных СТО на площадке отсутствуют, поэтому выбор мастерской нередко приходится делать вслепую.

На рисунке 1.3 представлена главная страница сайта.

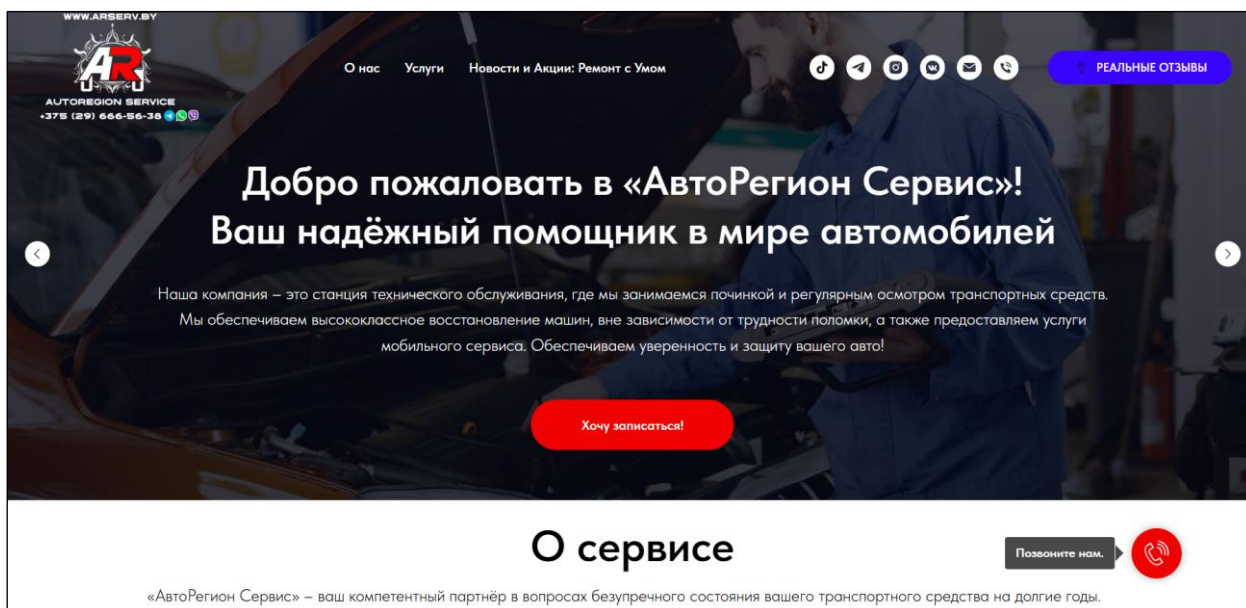


Рисунок 1.3 – Главная страница сайта «arserv.by»

Дизайн выполнен в тёмной цветовой гамме, что позволяет снизить утомляемость глаз при длительном просмотре, однако общее визуальное решение оценивается скорее как утилитарное, без излишних дизайнерских изысков. Главная страница даёт доступ к каталогу запчастей, разделу услуг, новостям и акциям.

Особенностью сайта является наличие специализированного раздела «Базар автозапчастей б/у», где представлены бывшие в употреблении детали с фотографиями и ценами – это расширяет ассортимент и привлекает покупателей, ищущих бюджетные варианты. Поиск по сайту как отдельный инструмент отсутствует, поэтому навигация по каталогу осуществляется через рубрикатор, что при большом количестве позиций может быть неудобно.

Авторизация и личный кабинет не реализованы, а для многих товаров указано «уточнять по телефону», что вынуждает пользователя переходить к офлайн-коммуникации и снижает оперативность покупки. Навигация в целом интуитивно понятна: главное меню позволяет быстро переходить между разделами, структура вложенности не вызывает затруднений. В разделе базара автозапчастей карточки товаров оформлены минималистично – фото, цена, краткое описание, что соответствует стандартам подобных площадок. К достоинствам можно отнести наличие узкоспециализированного раздела для б/у деталей и общую логичность структуры.

Однако отсутствие поиска, невозможность создания личного кабинета, а также не всегда высокая читаемость шрифтов (особенно на страницах с описаниями, где цвет текста сливается с фоном) требуют доработки. Кроме того, ориентация на телефонное уточнение деталей снижает доверие и создаёт дополнительные барьеры для клиента, привыкшего к полностью цифровому циклу заказа.

1.2 Постановка задачи

Исходя из анализа аналогичных решений, представленных в пункте 1.1, проектируемое программное средство должно устранить выявленные недостатки существующих веб-сайтов (отсутствие личного кабинета, перегруженность интерфейса, недостаточную интерактивность для клиентов и отсутствие инструментов для управления заказами со стороны персонала) и реализовать функционал, отвечающий современным требованиям к сервисным приложениям. На основе изучения сильных сторон аналогов (удобная навигация willgood.ru, контрастные кнопки призыва к действию fitauto.ru, наличие специализированных разделов arserv.by) были сформулированы следующие характеристики и функциональные возможности, которые должны быть заложены в разрабатываемое приложение:

- интуитивно понятная навигация, позволяющая пользователям быстро ориентироваться в приложении, находить необходимые услуги и записываться на ремонт без лишних затруднений, что повышает общий пользовательский опыт;
- продуманная цветовая палитра, обеспечивающая визуальную гармонию и формирующая узнаваемый стиль приложения; необходимо использовать не более двух–трёх акцентных цветов для кнопок и ключевых элементов, избегая излишней пестроты;
- оптимальный стиль, цвет и размер шрифта, гарантирующие хорошую читаемость на различных экранах; следует избегать ситуаций, наблюдаемых на arserv.by, где текст местами сливается с фоном, а размер шрифта недостаточен для комфортного восприятия;
- сбалансированное размещение контента на страницах без излишней захламлённости; в отличие от fitauto.ru, где крупный баннер, блоки услуг и карта создают визуальный шум, проектируемое приложение должно соблюдать принцип «воздуха» между элементами, равномерно распределяя текстовую и графическую информацию;
- возможность регистрации и авторизации для клиентов с сохранением персональных данных, истории записей и статусов заказов; данный функционал отсутствует во всех трёх рассмотренных аналогах, что делает его одним из ключевых преимуществ разрабатываемого решения;
- просмотр списка доступных услуг с актуальными ценами и подробным описанием; информация должна быть структурирована по категориям с возможностью быстрого перехода к записи;
- запись на ремонт с выбором даты, времени и конкретной услуги; процесс должен быть максимально простым, аналогично форме на fitauto.ru;
- отслеживание текущего статуса заказа в реальном времени с отображением этапов выполнения;
- для администратора: управление расписанием с возможностью просмотра свободных и занятых слотов, создания, редактирования и закрытия заказов, а также проверки наличия запчастей на складе с отображением актуальных остатков; эти функции позволят автоматизировать

рабочие процессы, которые в существующих аналогах решаются либо вручную, либо через отдельные учётные системы;

- для механика: просмотр персонального задания на текущий день;
- отметка о выполнении заказа с автоматическим обновлением статуса в системе и уведомлением администратора и клиента;
- поиск по названию для быстрого доступа к информации.

Для обеспечения бесперебойной работы проектируемого приложения предполагается использование современного стека технологий с акцентом на безопасность персональных данных и транзакций. Ролевая модель доступа разграничит функционал так, что каждый пользователь (клиент, администратор, механик) видит только необходимый блок информации. Ключевой особенностью станет встроенный модуль аналитики, позволяющий администратору в реальном времени отслеживать загрузку, оборачиваемость заказов. Особое значение в проекте отводится кроссплатформенности: приложение будет одинаково удобно работать на планшетах механиков в гараже, на ПК администратора и на смартфонах клиентов, причём для последних предусмотрен упрощённый сценарий записи в два клика с возможностью выбора конкретного мастера. Дополнительно система интегрируется с популярными мессенджерами и СМС-шлюзами для автоматического информирования о записи, готовности авто или изменении стоимости работ. В отличие от рассмотренных аналогов проектируемое приложение с первого экрана предлагает персонализированный дашборд с историей визитов, рекомендованными датами ТО и актуальными скидками, а также встроенный чат для отправки фото повреждений. В итоге будет создана целостная экосистема, где клиент всегда знает статус своего авто, администратор получает прозрачную аналитику, а механик — чёткое техническое задание и обеспечивает высокий уровень сервиса. Автоматизированная система уведомлений не только информирует клиента о смене статуса, но и запрашивает подтверждение на дополнительные работы, выявленные механиком в процессе диагностики, что исключает несогласованные траты.

Благодаря интеграции со складским учётом, приложение в реальном времени резервирует необходимые запчасти под конкретную заявку, автоматически предупреждая механика о дефиците позиций ещё до подъёма автомобиля на подъёмник.

Для администратора предусмотрен визуальный календарь с цветовой индикацией загрузки боксов и сотрудников, позволяющий перетаскивать заказы мышью и мгновенно перераспределять нагрузку при форс-мажорах.

Все действия пользователей логируются в защищённой базе, а критичные операции (изменение цены, удаление заказа, доступ к личным данным) требуют двухфакторной аутентификации, что сводит к нулю риск внутренних злоупотреблений.

2 Анализ требований к программному средству и разработка функциональных требований

Анализ требований – это этап в процессе разработки программного обеспечения, который определяет, что именно необходимо создать для удовлетворения потребностей пользователей и заказчиков. На этом этапе осуществляется сбор, документирование, уточнение, систематизация требований, что позволяет избежать недоразумений, ошибок и непоняток при дальнейшей разработке.

Разработка функциональных требований является важной частью этого процесса, так как именно эти требования определяют, что сможет делать пользователь, используя данное приложение. Функциональные требования формируют основу для проектирования архитектуры системы и последующей ее реализации, а также служат критерием для тестирования и оценки готовности продукта.

В данном разделе пояснительной записки будет представлено описание технических средств, которые были использованы в процессе разработки, также будет представлена архитектура системы.

2.1 Описание средств разработки

Для разработки приложения «Автосервис» были использованы современные технологии и инструменты, которые позволили обеспечить удобство использования, высокую производительность и надежность системы. В качестве основного языка программирования выбран C#, а платформой разработки – .NET Framework. Для построения клиентского приложения с графическим интерфейсом использована технология Windows Presentation Foundation (WPF), которая позволяет создавать функциональные и визуально привлекательные интерфейсы с поддержкой адаптивной компоновки и разделения логики от представления.

В качестве системы управления базами данных выбрана реляционная СУБД MS SQL Server. Она обеспечивает высокую производительность при работе с транзакциями, надежное хранение данных и гибкие возможности администрирования, что особенно важно для приложения, обрабатывающего информацию о клиентах, заказах, расписании работ и складских остатках. Взаимодействие приложения с базой данных реализовано с помощью фреймворка Entity Framework, который позволяет разработчикам работать с данными через .NET-объекты, минимизируя необходимость написания низкоуровневого SQL-кода и ускоряя разработку.

Для реализации функционала отправки уведомлений клиентам об изменении статуса заказа использована библиотека System.Net.Mail. Она предоставляет удобный интерфейс для создания и отправки электронных писем, что позволило оперативно информировать пользователей о подтверждении записи, начале ремонта, необходимости согласования дополнительных запчастей и завершении работ. Благодаря этому клиенты

всегда остаются в курсе происходящего, а администраторам не требуется тратить время на ручные звонки.

Далее будет более подробно рассмотрена каждая технология, используемая в проекте.

Язык программирования C# – современный объектно-ориентированный язык от Microsoft, являющийся основным для платформы .NET и позволяющий создавать веб-, мобильные и десктопные приложения благодаря высокой производительности, простому синтаксису и интеграции с библиотеками и фреймворками вроде ASP.NET и Entity Framework. Платформа .NET Framework предоставляет среду выполнения и набор инструментов для разработки, поддерживает несколько языков (включая C#), включает встроенные библиотеки для работы с базами данных, сетями и графикой, обеспечивает безопасность и высокую производительность через управление памятью, а также легко интегрируется со средой Visual Studio. Entity Framework – это объектно-ориентированная технология для работы с данными, которая абстрагируется от типа хранилища и позволяет оперировать сущностями и их связями (один-к-одному, один-ко-многим и многие-ко-многим), использует LINQ для выборки данных, а также включает Entity Data Model (концептуальный уровень, уровень хранилища и уровень маппинга), что упрощает разработку приложений. Windows Presentation Foundation (WPF) – технология для создания настольных приложений на Windows, входящая в .NET; она основана на векторной графике, использует язык разметки XAML для разделения логики и визуальных компонентов, поддерживает привязку данных, анимацию и графику, позволяя создавать современные, масштабируемые и интерактивные интерфейсы. MS SQL Server – популярная реляционная СУБД, обеспечивающая высокую производительность и масштабируемость, хранит данные в таблицах (строки и столбцы), предоставляет средства безопасности (шифрование, управление доступом, аудит) и интеграцию с Azure и Power BI, что делает её надёжным выбором для управления данными в современном бизнесе. Библиотека System.Net.Mail в .NET Framework – удобный инструмент для работы с электронной почтой через SMTP, позволяющий отправлять простые текстовые и HTML-сообщения с вложениями, поддерживает аутентификацию и шифрование, управление заголовками и несколькими получателями, легко интегрируется в проекты и помогает создавать безопасные решения для обмена сообщениями.

2.2 Спецификация функциональных требований

Программное средство должно предоставлять следующие функциональные возможности:

Для администратора:

- выполнять авторизацию;
- управлять расписанием;
- просматривать список всех заказов;

- создавать новый заказ (запись клиента) и редактировать существующий;
- закрывать заказ после выполнения работ;
- проверять наличие запчастей на складе при формировании или корректировке заказа;
- отправлять уведомления клиенту об изменении статуса заказа.

Для клиента:

- выполнять регистрацию и авторизацию;
- просматривать список доступных услуг и актуальные цены;
- выполнять поиск и фильтрацию услуг по категориям и стоимости;
- записываться на ремонт с выбором даты, времени и конкретной услуги;
- просматривать текущий статус своего заказа;
- просматривать историю своих обращений и заказов;
- вставлять отзывы о качестве обслуживания.

Для механика:

- выполнять авторизацию в системе;
- просматривать своё задание на текущий день;
- отмечать выполнение заказа;
- просматривать наличие запчастей на складе.

Чёткое разделение ролей позволяет автоматизировать ключевые бизнес-процессы автосервиса: от первичной записи и согласования заказа до его выполнения и получения обратной связи. Такой подход повышает удобство работы для каждой категории пользователей.

2.3 Диаграмма вариантов использования

Диаграмма вариантов использования (Use Case Diagram) — это графический инструмент для моделирования функциональности системы с точки зрения её пользователей. Она помогает выявить различные сценарии использования системы, включая взаимодействие между актерами (пользователями) и самой системой.

Преимущества диаграммы вариантов использования:

- Помогает увидеть все сценарии использования и понять взаимодействие с пользователями;
- Идентифицирует функциональные требования на основе сценариев использования;
- Служит средством общения между разработчиками, заказчиками и другими заинтересованными сторонами.

Use Case диаграмма для приложения «Автосервис» представлена в приложении А.

Для того чтобы детализировать поведение системы, необходимо рассмотреть каждого актера по отдельности. Начнём с роли «Клиент»,

которая является основной с точки зрения генерации дохода автосервиса. Клиент взаимодействует с приложением на всех этапах: от первого знакомства до получения закрывающих документов. Базовый вариант использования для клиента – регистрация в системе, которая позволяет создать постоянный профиль. После регистрации клиент может авторизоваться, подтверждая свою личность через логин и пароль. Следующий важный Use Case – просмотр каталога услуг с актуальными ценами, без которого невозможна осознанная запись. Сам процесс записи на ремонт включает выбор даты, времени, типа работ и желаемого механика. После создания заказа клиент получает уникальный идентификатор и возможность отслеживать статус выполнения. Для возвратных клиентов реализован Use Case «История обращений», где хранятся все предыдущие визиты. Завершающий вариант использования для клиента – оставление отзыва, который влияет на рейтинг механиков и сервиса в целом.

Второй актер – администратор, обладающий расширенными правами и отвечающий за операционное управление. Администратор взаимодействует с системой через отдельный интерфейс, где доступны все заказы в реальном времени. Ключевой Use Case для администратора – управление расписанием, включая перенос, отмену и блокировку слотов. Он также создаёт новые заказы (часто по телефону, от имени клиента) и редактирует существующие. По завершении работ администратор закрывает заказ, формируя акт выполненных услуг и финальную стоимость. В процессе ведения заказа администратор проверяет наличие запчастей на складе и при необходимости инициирует закупку. Отдельный важный вариант использования – отправка уведомлений клиентам о смене статуса, времени готовности или выявленных дополнительных работах. Администратор также может просматривать агрегированные отчёты по загрузке боксов и выручке, хотя это скорее вспомогательный Use Case.

Третий актер – механик, который работает непосредственно с автомобилями и нуждается в чётких, лаконичных интерфейсах. Каждое утро механик открывает в приложении список своих заданий на текущий день. На каждом этапе ремонта механик отмечает выполнение конкретных операций, фиксируя фактически потраченное время. Если в процессе диагностики обнаруживается необходимость в незапланированных деталях, механик использует вариант использования «Запросить дополнительные запчасти». Для быстрого поиска по каталогу или среди своих старых заданий реализован Use Case «Поиск по названию», экономящий время механика. В итоге все три актера связаны через общие объекты, а диаграмма вариантов использования наглядно показывает, кто и за что отвечает в приложении «Автосервис».

3 Проектирование программного средства

Проектирование программного обеспечения – это процесс разработки проекта, который включает в себя создание архитектуры и структуры

программного средства. Основной целью проектирования является определение внутренних характеристик системы и уточнение её внешних свойств с учетом исходных условий задачи на проект.

На этапе проектирования важно не только проанализировать сформулированные требования, но и выявить возможные ограничения и риски, которые могут повлиять на реализацию проекта. Нужно учитывать различные аспекты, такие как функциональность, производительность, безопасность и удобство использования. Это поможет создать более качественное и устойчивое программное средство, соответствующее ожиданиям пользователей.

Кроме того, проектирование включает в себя выбор архитектурных паттернов и технологий, которые будут использоваться в процессе разработки. Например, выбор между архитектурными стилями, такими как MVC, MVVM, монолитная или микросервисная архитектура, может существенно повлиять на гибкость и масштабируемость системы. Также необходимо определить, как будут организованы данные, какие базы данных будут использоваться и как будет обеспечено взаимодействие между компонентами.

Этап проектирования также подразумевает создание прототипов и макетов, которые помогают визуализировать интерфейс и функциональность системы. Это позволяет проводить ранние тестирования и получать обратную связь от пользователей, что может значительно улучшить конечный продукт.

Таким образом, проектирование программного средства является этапом, который формирует фундамент для успешной разработки и внедрения программного обеспечения, обеспечивая соответствие требованиям и ожиданиям конечных пользователей.

3.1 Проектирование архитектуры приложения

Архитектура программного обеспечения – это совокупность основных решений, касающихся структуры и организации программной системы. Она включает в себя выбор компонентов, их интерфейсов, определение взаимодействия между ними и интеграцию в более крупные системы. Ключевую роль в этой архитектуре играет архитектурный стиль, который задает общую структуру системы, включая элементы, их взаимодействие и способы соединения.

Для удовлетворения требований к качеству проектируемой системы применяются различные архитектурные паттерны. В данном проекте используется архитектурный шаблон Model-View-ViewModel (MVVM), который состоит из трех основных компонентов: модели (Model), модели представления (ViewModel) и представления (View).

Применение MVVM значительно улучшает модульность и возможность повторного использования кода, а также упрощает разработку и тестирование. Этот шаблон помогает разделить бизнес-логику приложения и

пользовательский интерфейс, что делает систему более гибкой и масштабируемой, позволяя адаптироваться к изменениям требований и легко расширяться в будущем.

На рисунке 3.1 представлена структура архитектуры MVVM.

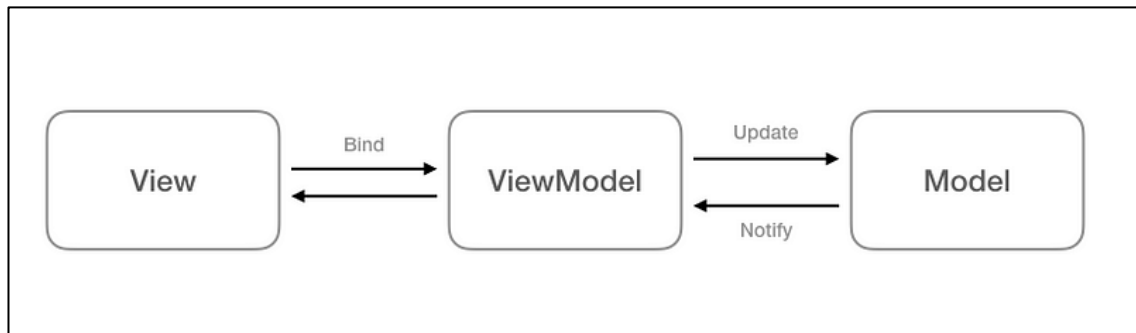


Рисунок 3.1 – Структура MVVM

Модель (Model) описывает данные, используемые в приложении. В ней может содержаться логика, связанная с этими данными, например, валидация. Однако модель не должна содержать логику, касающуюся отображения данных или взаимодействия с пользовательским интерфейсом. Часто модель реализует интерфейс `INotifyPropertyChanged`, что позволяет автоматически отслеживать изменения свойств и облегчает привязку к представлению, хотя сама модель не взаимодействует напрямую с ним.

Представление (View) определяет визуальный интерфейс, который пользователь использует для взаимодействия с приложением. Важно проектировать пользовательский интерфейс так, чтобы он был удобным и приятным для пользователя. Представление не обрабатывает события непосредственно, а выполняет действия через команды.

Модель представления (ViewModel) связывает модель и представление с помощью механизма привязки данных. При изменении значений свойств модели, используя `INotifyPropertyChanged`, данные в представлении автоматически обновляются, хотя модель и представление не связаны напрямую. ViewModel также содержит логику для получения и обновления данных в модели.

Кроме того, в проекте применяются паттерны для работы с базой данных: Repository и Unit of Work.

Паттерн Unit of Work позволяет упростить работу с различными репозиториями и обеспечивает уверенность в том, что все репозитории используют один и тот же контекст данных. Он следит за тем, чтобы все изменения проходили через единую точку, что позволяет либо подтверждать все изменения сразу, либо откатывать их в случае необходимости. Это гарантирует, что все операции будут выполнены успешно или отменены целиком, что особенно важно для поддержания целостности данных.

Паттерн Repository служит для абстрагирования от конкретных подключений к источникам данных, с которыми работает программа. Он представляет собой промежуточное звено между классами, непосредственно

взаимодействующими с данными, и остальной частью приложения. Благодаря этому паттерну можно легко изменять источник данных или способ доступа к ним, не затрагивая остальную логику приложения. Это повышает гибкость и упрощает тестирование, так как можно подменять реализацию репозитория на мок-объекты.

3.2 Проектирование структуры базы данных

База данных – это организованная совокупность взаимосвязанных данных, которая служит хранилищем динамически обновляемой информации. Она позволяет эффективно управлять данными, обеспечивая их хранение, извлечение, обновление и обработку.

Система управления базами данных – это программная реализация технологии хранения, извлечения, обновления и обработки данных в базе данных, которая позволяет пользователям и приложениям взаимодействовать с данными.

В данном проекте для создания и управления базой данных использовалась реляционная СУБД Microsoft SQL Server, которая предоставляет мощные средства для работы с данными и обеспечивает высокую производительность и безопасность. База данных для приложения «Магазин автозапчастей» состоит из 11 таблиц: Users, Roles, Orders, OrderStatuses, Reviews, Services, MechanicRequests, SpareParts, PasswordResetCodes, Notifications.

Назначение таблиц:

- В таблице Users хранится информация о пользователях (ID, ФИО, email, хеш пароля, телефон, роль, дата создания);
- В таблице Roles хранятся возможные роли пользователей (например, клиент, механик, администратор);
- В таблице Orders хранится информация о заказах (клиент, механик, даты, статус, стоимость, отзыв, оценка, год автомобиля);
- В таблице OrderStatuses содержатся возможные статусы заказов;
- В таблице Reviews хранятся отзывы на заказы с рейтингом, текстом и фото;
- В таблице Services хранится информация об услугах (название, описание, цена, длительность, категория);
- В таблице MechanicRequests содержится информация о запросах механиков на запчасти (какая деталь, количество, статус);
- В таблице SpareParts хранятся автозапчасти (название, артикул, количество на складе, цена);
- В таблице PasswordResetCodes хранятся коды для сброса пароля с временем действия и отметкой использования;
- В таблице Notifications содержатся уведомления для пользователей (сообщение, прочитано ли, дата).

Для обеспечения целостности и согласованности данных в базе данных используются внешние ключи и индексы. Это помогает оптимизировать

запросы и поддерживать связи между таблицами (например, Orders связан с Users (ClientId, MehandId), Reviews связан с Orders).

Кроме того, база данных может быть расширена за счет добавления новых таблиц или новых полей, если возникнет необходимость в хранении дополнительной информации.

Использование Microsoft SQL Server также обеспечивает надежное резервное копирование и восстановление данных, что критически важно для защиты информации в случае сбоев или потерь данных.

В приложении В представлена логическая модель базы данных AutoService.

Между таблицами существуют связи. Есть два вида: один к одному, примером которой является связь между Users и PasswordResetCodes по полю UserId (если предположить, что у каждого пользователя может быть только один активный код сброса пароля). Также присутствует связь один ко многим, примером которой являются связи между Users и Orders (у каждого пользователя может быть много заказов, но каждый заказ принадлежит только одному пользователю), между Roles и Users (одна роль может быть у многих пользователей), между Orders и MechanicRequests (один заказ может содержать несколько запросов на запчасти), между SpareParts и MechanicRequests (одна деталь может запрашиваться в нескольких заказах). Связь многие ко многим в данной базе данных явно не используется, но может быть реализована при необходимости (например, через таблицу связи OrderServices).

Для работы с базой данных в приложении используется библиотека Entity Framework 6 (в проекте подключена версия 6.5.1 через NuGet). Она относится к классическому стеку Entity Framework для .NET (пространство имён System.Data.Entity) и позволяет работать с данными как с обычными классами C# через контекст DbContext и коллекции DbSet<T>. По сравнению с прямым использованием ADO.NET, в прикладном коде, как правило, не пишут SQL вручную: выборки и изменения формулируются с помощью LINQ, а преобразование запросов в SQL выполняет сама библиотека Entity Framework.

В Entity Framework выделяют подходы Model-First, Database-First и Code-First. В данном проекте данные описываются классами сущностей в коде (например, User, Order, Service, SparePart и др.), для них задан контекст AutoServiceContext с соответствующими DbSet, а отдельные правила сопоставления с таблицами задаются в OnModelCreating. Таким образом, модель задаётся в коде приложения (подход в духе Code-First). Для синхронизации схемы с моделью используются Code First Migrations: при подходящей конфигурации база может создаваться или обновляться при запуске; при этом для баз, созданных ранее вручную (например, SQL-скриптом) без истории миграций EF, предусмотрен безопасный режим без принудительного пересоздания уже существующих таблиц, а донастройка структуры выполняется служебной логикой приложения. Начальные и демонстрационные данные задаются при запуске через DbSeedService.

Такой подход упрощает поддержку и развитие приложения: изменения в основном вносятся на уровне классов модели и LINQ-запросов, а не через ручное сопровождение каждого SQL-оператора к таблицам. Это снижает связанность бизнес-логики с деталями конкретной СУБД и ускоряет адаптацию к изменениям требований.

Использование Entity Framework 6 в связке с моделью в коде и миграциями позволяет удобно и предсказуемо работать с базой данных и сохранять контроль над схемой и данными в процессе разработки и сопровождения приложения.

3.3 Структура проекта

Структура проекта представлена на рисунке 3.3.

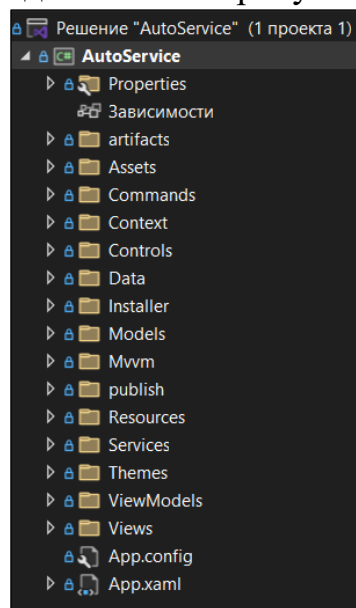


Рисунок 3.3 – Структура проекта

- Папка Context содержит класс контекста Entity Framework, который отвечает за подключение к базе данных, конфигурацию модели и связей между сущностями, выполнение запросов, сохранение изменений, отслеживание состояний объектов и работу с транзакциями;
- Папка Models включает классы сущностей базы данных (описание таблиц и их полей);
- Папка ViewModels содержит промежуточный слой между моделями и представлениями: обработку данных, команд и бизнес-логики UI;
- Папка Views хранит XAML-представления, определяющие пользовательский интерфейс приложения; здесь же находится главное окно MainWindow.xaml;
- Папка Resources содержит ресурсы приложения (изображения, стили, словари ресурсов, курсоры и др.);
- Папка Controls включает пользовательские элементы интерфейса, созданные для переиспользования кода и упрощения сопровождения;

- Папка Assets хранит статические файлы проекта, включая шрифты и другие медиа-ресурсы;
- Файл App.config содержит параметры конфигурации приложения;
- Файл App.xaml определяет общие ресурсы и стартовые настройки WPF-приложения.

Таким образом я наглядно показал, как выглядит структура моего проекта.

3.4 Диаграмма классов

При создании программного продукта для планирования задач и ведения проектов, диаграмма классов выступает важным средством для моделирования и проектирования системы. Она визуализирует организацию системы, отображая классы, их характеристики и связи между ними.

В ходе разработки программного обеспечения для организации задач и управления проектами, диаграмма классов способствует выявлению ключевых классов, их свойств и методов. Она предоставляет возможность увидеть структуру данных, применяемых в приложении, а также взаимосвязи между классами. Помимо этого, диаграмма классов способствует более глубокому пониманию концептуальной модели системы и создает фундамент для дальнейшей разработки.

Диаграмма классов отображает статическую структуру системы через классы, их атрибуты и методы, а также связи между ними. Основные типы связей включают ассоциацию (простая связь), агрегацию и композицию (отношения «часть-целое» с разной степенью жёсткости), наследование (стрелка с пустым треугольником) и реализацию интерфейса. Для каждой связи указывается множественность (например, 1, 0.., 1..), определяющая количество экземпляров, а модификаторы видимости (+ public, - private, # protected) задают уровень доступа к атрибутам и операциям. В приложении «Автосервис» базовый класс «Пользователь» порождает наследников «Клиент», «Администратор» и «Механик», а класс «Заказ» связывается с ними через ассоциации и композицию с позициями заказа.

Диаграмма классов представлена в приложении Б.

4 Реализация программного средства

На данном этапе осуществляется разработка программного обеспечения, реализующего предложенное техническое решение, а также сборка программного средства. В процессе работы разрабатываются программные интерфейсы для взаимосвязи между классами, методами и функциями.

Важным шагом является документирование классов, их атрибутов и методов, что позволяет обеспечить ясность и удобство в дальнейшем

использовании. Программирование включает написание текстов программ и отладку отдельных модулей проекта.

В результате выполнения этого этапа будет создано полноценное функционирующее программное средство.

В приложении реализовано множество классов, каждый из которых выполняет свою определенную задачу, что обеспечивает модульность и гибкость системы. Однако в данном разделе будет уделено внимание лишь некоторым из них, которые непосредственно отвечают за реализацию паттерна MVVM и обеспечивают связь с базой данных.

Паттерн MVVM является основой архитектуры нашего приложения, так как он позволяет четко разделить логику представления и бизнес-логику.

Классы, отвечающие за связь с базой данных, обеспечивают надежное и эффективное управление данными. Они реализуют функции, такие как выполнение запросов, обработка транзакций и управление соединениями с базой данных. Это позволяет приложению динамически загружать и сохранять информацию, обеспечивая актуальность данных.

Эти классы играют ключевую роль в организации структуры приложения, обеспечивая взаимодействие между пользовательским интерфейсом и данными.

4.1 Гибкость системы

4.1.1 Классы для реализации паттерна MVVM

Как уже было сказано выше приложение «Автосервис» реализует паттерн MVVM. Как можно понять из названия, для его осуществления должны быть созданы классы, каждый из которых отвечает за свою часть:

- Model (представляет данные и бизнес-логику приложения, описывает таблицы базы данных).
- View (визуальный интерфейс пользователя, отвечает за отображение данных и взаимодействие с пользователем).
- ViewModel (служит промежуточным слоем между моделью и представлением, обрабатывает данные и команды, поступающие из представления).

Использование паттерна MVVM будет показано на примере страницы авторизации. В листинге 4.1 представлен фрагмент кода из представления (view) страницы авторизации, на котором отображена xaml-разметка, которая предназначена для создания кнопки для входа. На данном примере показано использование механизма привязки (binding) команды RegisterCommand к кнопке регистрации, тем самым продемонстрирована связь представления (view) с моделью представления (viewmodel). Это позволяет кнопке или другому элементу управления в представлении выполнять команду, определенную в модели представления, обеспечивая четкое разделение логики и интерфейса.

В приложение Г будет вынесен полный код данной страницы для подробного ознакомления.

```
<Button x:Name="btnRegister" Grid.Row="6" Grid.Column="1" Margin="5"
Command="{Binding RegisterCommand}"/>
```

Листинг 4.1 – Фрагмент кода xaml, демонстрирующий кнопку с привязкой к команде

Код модели представления (viewmodel) для данной страницы будет вынесен в приложение Г. ViewModel выступает посредником между View и Model, предоставляя данные для отображения и обрабатывая действия пользователя. Этот класс реализует интерфейс INotifyPropertyChanged для динамического обновления UI, вызывая событие PropertyChanged при изменении свойств. Он получает данные из Model, преобразует их для View и выполняет логику, но не модифицирует Model напрямую, а вызывает её методы. В данном классе действия пользователя обрабатываются через команды (Commands).

Перейдем к последней составляющей MVVM, которая отвечает за данные на нашей странице. В данном случае в коде команды RegisterCommand, происходит обращение к данным о пользователях через модели (model) User, который содержит информацию о пользователях и их реквизитах для входа соответственно. По сути своей модель данных представляет из себя обычный класс, который представляет данные приложения. Она предоставляет интерфейс (методы и свойства) для доступа и изменения данных, может содержать логику для валидации и преобразования данных, а также для сохранения и извлечения данных из хранилища. Преимущества включают в себя повторное использование в разных частях приложения или даже в разных приложениях, лёгкую тестируемость, разделение ответственности между компонентами MVVM и гибкость при изменениях или расширениях без влияния на UI или ViewModel. Этот класс не зависит от конкретной реализации пользовательского интерфейса. Это позволяет использовать его в разных приложениях и на разных страницах, а также быть легко тестируемым независимо от UI-логики, что делает разработку более эффективной и поддерживаемой.

В листинге 4.2 представлена модель User.

```

public class User
{
    public int Id { get; set; }
    public string FullName { get; set; }
    public string Email { get; set; }
    public string PasswordHash { get; set; }
    public string Phone { get; set; }
    public int RoleId { get; set; }
}

```

Листинг 4.2 – Модель User

В результате анализа реализации паттерна MVVM на примере страницы авторизации приложения становится очевидным, как эффективно организованы компоненты системы. Классы, отвечающие за модель, представление и модель представления, обеспечивают четкое разделение логики и интерфейса, что способствует упрощению разработки и тестирования.

4.1.2 Классы для работы с базой данных

Для работы с базой данных, взаимодействие с которой происходит при помощи специальной библиотеки Entity Framework, также требует наличия специальных классов. Данные классы представляют из себя класс сущности (по сути, это и есть модель) и класс контекста данных, который используется для взаимодействия непосредственно с самой базой данных. Класс контекста должен наследоваться от класса DbContext. В реализации данного класса с помощью DbSet указываются какие таблицы нужно создать в базе данных. Также следует реализовать в этом классе один перегруженный метода OnModelCreating. OnModelCreating используется для конфигурации модели данных при создании базы данных. Здесь можно задать правила отображения (mapping) между классами модели и таблицами базы данных, а также определить связи между сущностями, ограничения и индексы.

Стоит сказать также, что AutoServiceContext реализует паттерны Unit of Work и Repository, что значительно упрощает управление доступом к данным и взаимодействие с базой данных. Паттерн Unit of Work обеспечивает согласованное выполнение операций с данными, позволяя отслеживать изменения в объектах и координировать их сохранение, что минимизирует риски нарушения целостности данных и упрощает управление транзакциями. В свою очередь, паттерн Repository служит абстракцией для доступа к данным, скрывая детали реализации хранения и запроса информации. Он предоставляет удобный интерфейс для выполнения операций CRUD, что облегчает тестирование бизнес-логики независимо от конкретной реализации базы данных. Таким образом, класс AutoServiceContext не только структурирует взаимодействие с данными, но и способствует чистоте архитектуры приложения, делая его более удобным для сопровождения и расширения. В приложении Е приводится код реализации данного класса.

4.2 Реализация функционала приложения

В данном подразделе будут представлены некоторые методы, обеспечивающие функционал приложения: метод для добавления и удаления товара из избранного, метод для добавления и удаления товара из корзины, метод для регистрации пользователя в приложении, а также метод для формирования заказа и отправки его на рассмотрение администратору. Диаграмма последовательности находится в приложении 3, блок-схема алгоритма записи в приложении И.

4.2.1 Метод авторизации пользователя в приложении

Код данного метода представлен в приложении Ж. В нём выполняется нормализация введённого адреса электронной почты и пароля. Далее через фабрику единицы работы создаётся контекст доступа к данным (описанный в разделе 4.1.2 класс `AutoServiceContext` в составе `UnitOfWork`). В таблице `Users` выполняется поиск пользователя по e-mail с подгрузкой связанной записи роли (`Roles`). Вычисляется хэш введённого пароля и сравнивается с сохранённым `PasswordHash`. При успешном совпадении возвращается объект пользователя с ролью; иначе авторизация не выполняется. На уровне модели представления окна входа текущий пользователь сохраняется в ресурсах/сессии приложения (например, `App.CurrentUser`), после чего выполняется переход к главному окну с панелью, соответствующей роли (клиент, администратор, механик).

4.2.2 Метод регистрации клиента в приложении

Код данного метода представлен в приложении Ж; полная логика полей и команд окна регистрации вынесена в приложение Ж (модель представления `RegisterViewModel`). Сначала проверяется заполненность обязательных полей (ФИО, e-mail, пароль), корректность e-mail и телефона (если указан), совпадение пароля и подтверждения. Затем вызывается сервис `AuthService.RegisterClientAsync`, который открывает `AutoServiceContext` и проверяет, что пользователя с таким e-mail ещё нет в таблице `Users`. При дубликате регистрация прерывается с ключом ошибки для локализованного сообщения. Иначе из таблицы `Roles` выбирается роль «Клиент» / «Client», создаётся новая запись пользователя с хэшем пароля, идентификатором роли и датой создания, запись добавляется в `Users` и сохраняется вызовом `SaveChanges()`. В отличие от примера с отдельной таблицей реквизитов, в данном приложении учётные и контактные данные клиента хранятся в самой сущности пользователя. После успешного сохранения пользователю показывается информационное сообщение и закрывается окно регистрации.

4.2.3 Методы клиента: просмотр услуг и цен, запись на ремонт

Код представлен в приложении Ж (сервис ClientPanelService, модель представления ClientPanelViewModel). Для отображения каталога выполняется обращение к AutoServiceContext: из таблицы Services выбираются услуги с возможной фильтрацией по подстроке в названии; для витрины дополнительно формируются карточки услуг и запчастей по таблицам Services и SpareParts с отображением цен. Для записи на ремонт клиент выбирает дату; свободные интервалы для выбранного дня рассчитываются сервисом слотов (AppointmentSlotService) с учётом уже занятых записей в Orders. После выбора времени и одной или нескольких услуг вызывается BookAppointmentAsync: проверяется, что слот не занят; из OrderStatuses берётся статус «Новый»; в таблицу Orders добавляется заказ с привязкой к текущему клиенту (ClientId), датой оформления, датой записи (AppointmentDate) и предварительной суммой; после SaveChanges() для каждой выбранной услуги в таблицу OrderServices добавляются строки с количеством, идентификатором услуги и ценой на момент записи (PriceAtTime), снова вызывается SaveChanges(). Через DataSyncService оповещаются подписчики об изменении заказов; администратору может быть создано уведомление в таблице Notifications. В интерфейсе очищается список выбранных услуг и обновляется список заказов клиента.

4.2.4 Отслеживание статуса заказа и обратная связь клиента

Код представлен в приложении Ж. Список заказов текущего пользователя загружается методом GetClientOrdersAsync: из Orders отбираются записи с ClientId, равным идентификатору авторизованного клиента, с подгрузкой связанного статуса (OrderStatuses), сортировка по дате заказа. Текущий этап ремонта отображается по полю статуса заказа. Дополнительно реализована возможность оставить отзыв по выполненному заказу: при вызове команды проверяется, что у заказа статус «Выполнен»; затем открывается диалог отзыва, работающий с таблицей Reviews (детали — в приложении Е). Изменения статусов и заказов, инициированные администратором или механиком, клиент видит после обновления коллекции при событиях DataSyncService.OrdersChanged.

4.2.5 Методы администратора: расписание, заказы, склад, уведомления о смене статуса

Код представлен в приложении Ж (сервис AdminPanelService, соответствующие ViewModel панели администратора). Расписание строится выборкой из Orders с фильтром по AppointmentDate на выбранный день либо отображением всех записей; к заказам подгружаются клиент, механик, статус и состав услуг; итоговая стоимость может пересчитываться через OrderPricingService. Создание и правка заказа выполняются через диалоги (AddAppointmentDialog, OrderEditDialog и др.): изменения сохраняются в

AutoServiceContext с вызовом SaveChanges() и рассылкой событий синхронизации. Управление номенклатурой услуг и запчастей осуществляется обращениями к таблицам Services и SpareParts; для контроля остатков используется поле количества на складе. Отправка сообщения клиенту реализована методом NotifyClientAsync: создаётся запись в Notifications, вызывается NotificationService.CreateNotification, который сохраняет уведомление в базе и при наличии e-mail у пользователя инициирует отправку письма через EmailService. Аналогично при отмене записи администратором клиенту создаётся уведомление с текстом об отмене. Рассмотрение заявок механика на запчасти (MechanicRequests) с утверждением или отклонением связано со списанием или возвратом количества на складе и при необходимости с автоматическим изменением статуса заказа (например, «Ожидает запчасти» / «В работе») с дополнительными уведомлениями клиенту и механику.

4.2.6 Методы механика: задания на день, выполнение заказа, поиск, запрос запчастей

Код представлен в приложении Ж (сервис MechanicPanelService, модель представления MechanicPanelViewModel). Список заданий формируется запросом к Orders, отфильтрованным по MechanicId текущего пользователя, с подгрузкой клиента, статуса и услуг; опционально включается режим «только сегодня» по AppointmentDate, а также текстовый поиск по ФИО клиента или названию услуги. Команда «начать работы» (StartWorkAsync) переводит заказ из статуса «Подтвержден» в «В работе» после проверок, сохраняет изменения и создаёт уведомление клиенту. Команда «завершить» (FinishWorkAsync) при статусе «В работе» устанавливает статус «Выполнен», пересчитывает TotalCost, при необходимости формирует квитанцию, уведомляет клиента. Запрос дополнительных запчастей открывает диалог RequestSparePartsDialog; при успешном создании заявки добавляются записи в MechanicRequests (и изменения на складе — по логике диалога), администратору отправляется уведомление; интерфейсы обновляются через DataSyncService. Справочник доступных на складе позиций с положительным остатком выводится отдельным запросом к SpareParts с поиском по названию или номеру детали. Механик также имеет возможность просматривать историю выполненных работ по каждому автомобилю, что помогает быстрее диагностировать повторяющиеся неисправности без повторного изучения бумажных архивов.

Все действия механика логируются во временном слое, что позволяет администратору в реальном времени видеть фактическую занятость сотрудника и при необходимости корректировать расписание.

5 Тестирование, проверка работоспособности приложения и анализ результатов

В данном разделе представлено тестирование спроектированной программной системы, направленное на подтверждение её работоспособности в соответствии с требованиями технического задания. Описаны результаты тестирования и анализ этих результатов. Рассмотрена реакция системы на исключительные и некорректные ситуации при работе клиента и администратора. Цель раздела — предоставить обоснование того, что разработанная ПС удовлетворяет заявленным требованиям и пригодна к эксплуатации.

5.1 Тестирование функций окна регистрации

Первым тестом выполняется попытка регистрации при незаполненном одном из обязательных полей (например, поле ФИО или e-mail оставлено пустым). Ожидаемый результат — вывод предупреждения о необходимости заполнить обязательные поля. Результат теста представлен на рисунке 5.1.

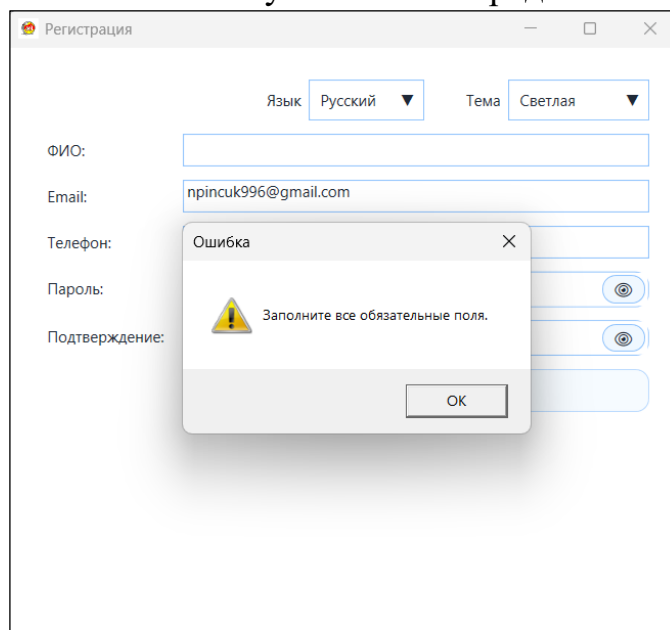


Рисунок 5.1 – Тест на пустое обязательное поле при регистрации

Вторым тестом выполняется попытка зарегистрировать клиента с адресом электронной почты, который уже есть в базе данных. Ожидаемый результат — сообщение о том, что пользователь с таким e-mail уже существует, регистрация не выполняется. Данная проверка реализована на уровне бизнес-логики приложения: перед вставкой новой записи выполняется выборка по полю Email, и при обнаружении совпадения транзакция откатывается без изменений в базе данных.

Такой подход предотвращает дублирование учётных записей и сможет восстановить доступ к профилю по Email.

Результат теста представлен на рисунке 5.2.

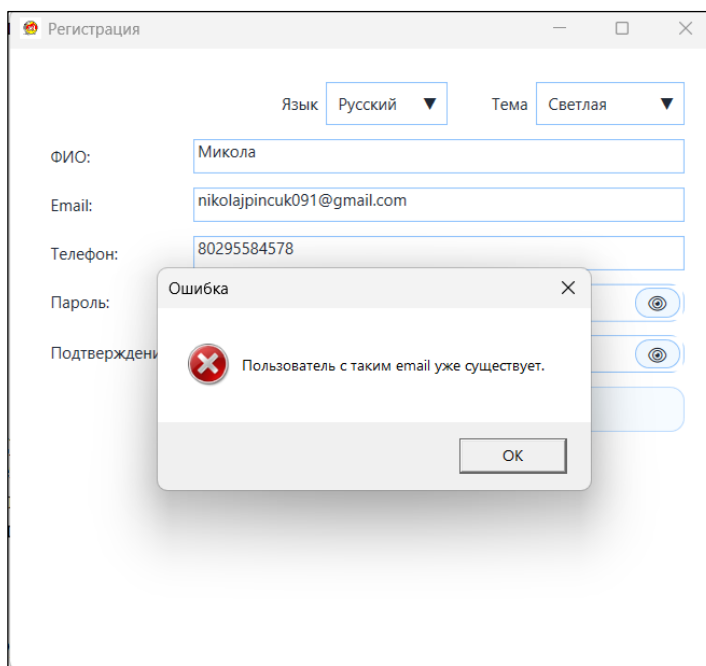


Рисунок 5.2 – Тест на уже зарегистрированный адрес электронной почты

Вывод: в окне регистрации проверка вводимых данных выполняется корректно, пользователь получает понятные сообщения об ошибках.

5.2 Тестирование функций окна авторизации

Первым тестом выполняется попытка входа при пустом поле пароля (или при пустом e-mail вместе с паролем, в зависимости от сценария). Ожидаемый результат — предупреждение о необходимости заполнить обязательные поля. Результат теста представлен на рисунке 5.3.

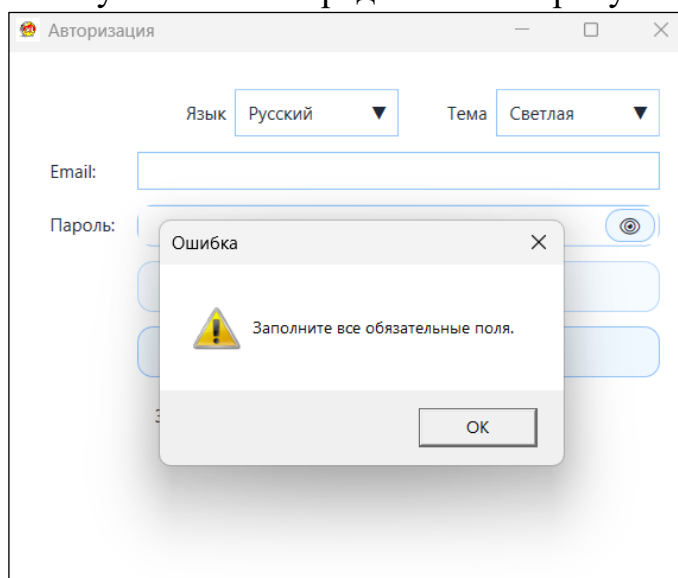


Рисунок 5.3 – Тест на незаполненные поля входа (e-mail и/или пароль)

Вторым тестом выполняется ввод произвольного e-mail и пароля для учётной записи, отсутствующей в системе, либо неверного пароля для

существующего пользователя. Ожидаемый результат – сообщение об ошибке входа. Результат теста представлен на рисунке 5.4.

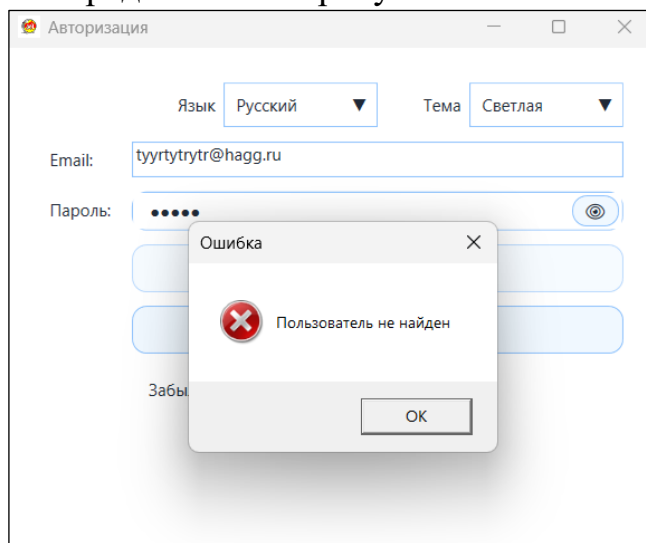


Рисунок 5.4 – Тест на несуществующую учётную запись или неверный пароль

Вывод: в окне авторизации проверка вводимых данных и результат обращения к базе обрабатываются корректно, пользователь информируется об ошибке.

5.3 Тестирование функций администратора

В данном подразделе проверяется работа администратора с перечнем услуг и ценами.

Первым тестом при добавлении или редактировании услуги в диалоге в поле цены вводится нечисловое значение. Ожидаемый результат — отказ сохранения и сообщение о некорректной цене. Результат теста представлен на рисунке 5.5.

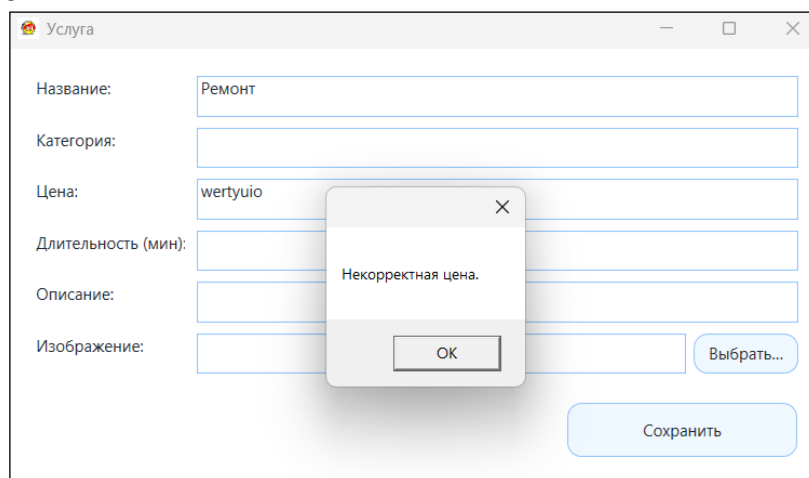


Рисунок 5.5 – Тест сохранения услуги с некорректным значением цены

Вторым тестом проверяется удаление услуги, которая уже связана с заказами (имеются записи в составе заказа). Ожидаемый результат —

удаление не выполняется, выводится предупреждение о невозможности удалить услугу, используемую в заказах. Результат теста представлен на рисунке 5.6.

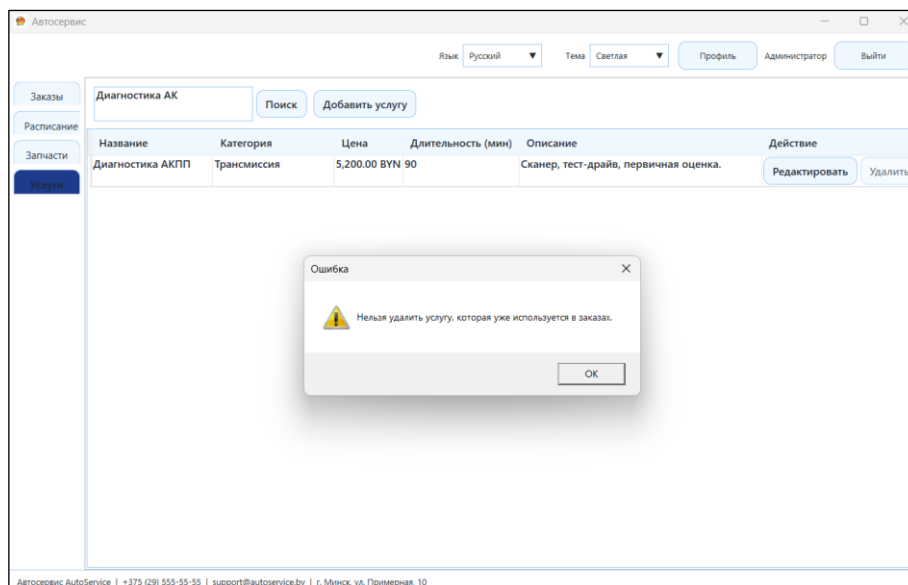


Рисунок 5.6 – Тест запрета удаления услуги, задействованной в заказах

Вывод: функции ведения справочника услуг устойчивы к некорректному вводу и к операциям, нарушающим целостность данных по заказам.

По результатам тестирования подтверждены базовые аспекты работоспособности и устойчивости приложения к ошибочным действиям пользователя. Проверки окна регистрации и окна входа показали корректную валидацию и информирование об ошибках. Проверки сценариев администратора при работе со справочником услуг подтвердили корректную обработку некорректной цены и запрет удаления записей, связанных с заказами, что соответствует требованиям надёжности и целостности данных автоматизированной системы учёта автосервиса. В ходе тестирования сценариев механика также была подтверждена корректная фильтрация заданий по идентификатору текущего сотрудника, исключающая доступ к чужим заказам.

Валидация поля «цена услуги» при вводе буквенных символов и отрицательных чисел в окне администратора завершилась штатным сообщением об ошибке без вылета приложения.

Итоговая проверка сценария восстановления пароля клиента через Email подтвердила, что временный токен генерируется, отправляется и принимается системой, а старый пароль после сброса становится недействительным, что закрывает требование к безопасному восстановлению доступа.

6 Руководство по использованию

В данном разделе описан порядок работы с программой «Автосервис» для пользователей с ролями клиент, администратор и механик. Интерфейс настольного приложения (WPF): после входа открывается главное окно с панелью, соответствующей роли. В верхней части главного окна доступны переключение языка и цветовой темы, кнопка «Профиль» (или аналог по локализации), отображение ФИО текущего пользователя и кнопка выхода из системы. Для удобства заданы сочетания клавиш: Ctrl+P – профиль, Ctrl+L – выход, F5 – обновление данных на активной панели. В нижней части окна выводится контактная строка автосервиса.

6.1 Регистрация и авторизация

При запуске приложения открывается окно авторизации. Для входа необходимо указать адрес электронной почты и пароль, затем подтвердить вход кнопкой входа в систему (рисунок 6.1).

Рисунок 6.1 – Окно авторизации

Если учётной записи ещё нет, в окне авторизации следует нажать кнопку перехода к регистрации. Откроется окно регистрации (рисунок 6.2), где нужно заполнить ФИО, e-mail, телефон (по желанию), пароль и подтверждение пароля. При неверном формате данных или несовпадении паролей приложение выводит предупреждение, регистрация не завершается. Если пользователь с таким e-mail уже существует, выводится соответствующее сообщение. После успешной регистрации отображается информационное сообщение об успешном создании учётной записи; для дальнейшей работы следует войти в систему, используя указанные при регистрации e-mail и пароль.

Рисунок 6.2 – Окно регистрации

В окне авторизации и регистрации также можно сменить язык интерфейса и тему оформления до входа в главное окно.

6.2 Использование приложения в роли клиента

После успешного входа пользователь с ролью «Клиент» попадает в главное окно приложения, в области которого отображается панель клиента с вертикальным меню слева (рисунок 6.3).

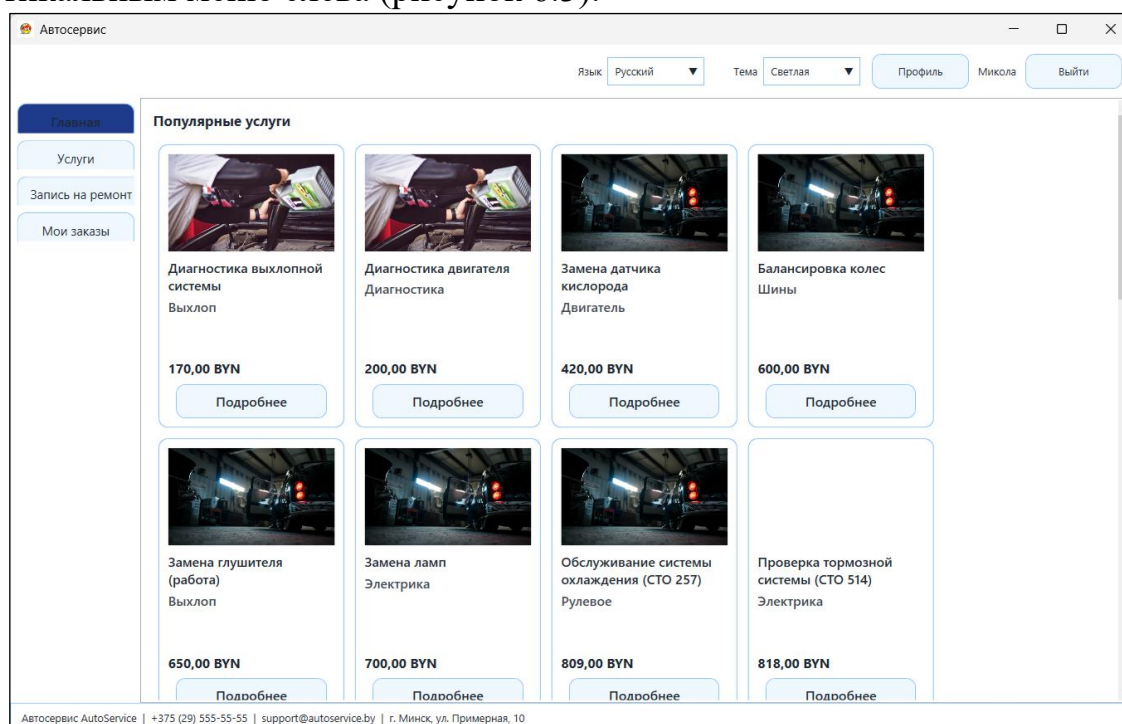


Рисунок 6.3 – Главное окно приложения (роль «Клиент»)

Навигация по функциям клиента осуществляется вкладками слева: «Главная», «Услуги», «Запись на ремонт», «Мои заказы».

Вкладка «Главная». Отображаются витрины популярных услуг и запчастей с изображением, названием, краткой подписью и ценой в BYN. Кнопка «Подробнее» открывает окно с подробной информацией по выбранной услуге или запчасти (рисунок 6.4).

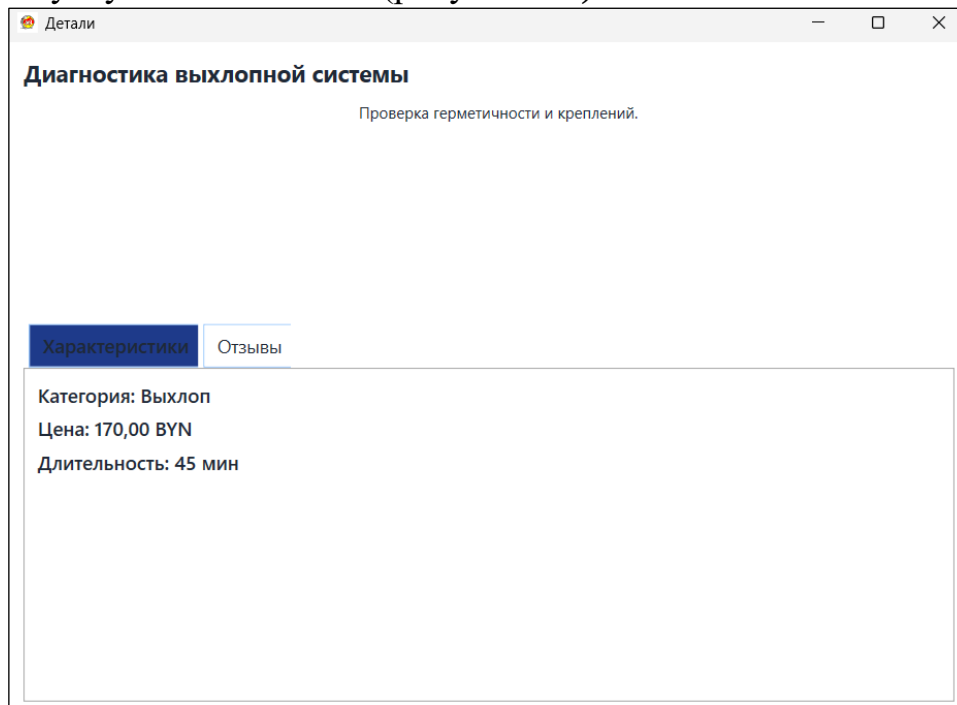


Рисунок 6.4 – Окно подробной информации об услуге или запчасти

Вкладка «Услуги». Выводится таблица доступных услуг: наименование, категория, цена, длительность в минутах. Поле поиска и кнопка «Поиск» позволяют отфильтровать услуги по названию. Кнопка в строке таблицы «Выбрать» (подпись может зависеть от языка) добавляет услугу в список выбранных для будущей записи на ремонт.

Вкладка «Запись на ремонт». Необходимо:

- Выбрать дату в календаре – после этого загрузится список свободных интервалов времени;
- Выбрать время из списка;
- убедиться, что в блоке выбранных услуг есть хотя бы одна услуга (услуги добавляются на вкладке «Услуги»);
- нажать кнопку оформления записи (запись на ремонт).

Отображается предварительная сумма выбранных услуг. Если не выбраны дата, время или услуги, приложение сообщит об ошибке. При успешной записи создаётся заказ со статусом в очереди обработки; при необходимости администратор получает уведомление о новой записи.

Вкладка «Мои заказы». Таблица содержит дату и время записи, текущий статус заказа, стоимость и кнопку оставления отзыва (рисунок 6.5). Отзыв доступен для заказов в статусе «Выполнен»; при нажатии открывается диалог отзыва.

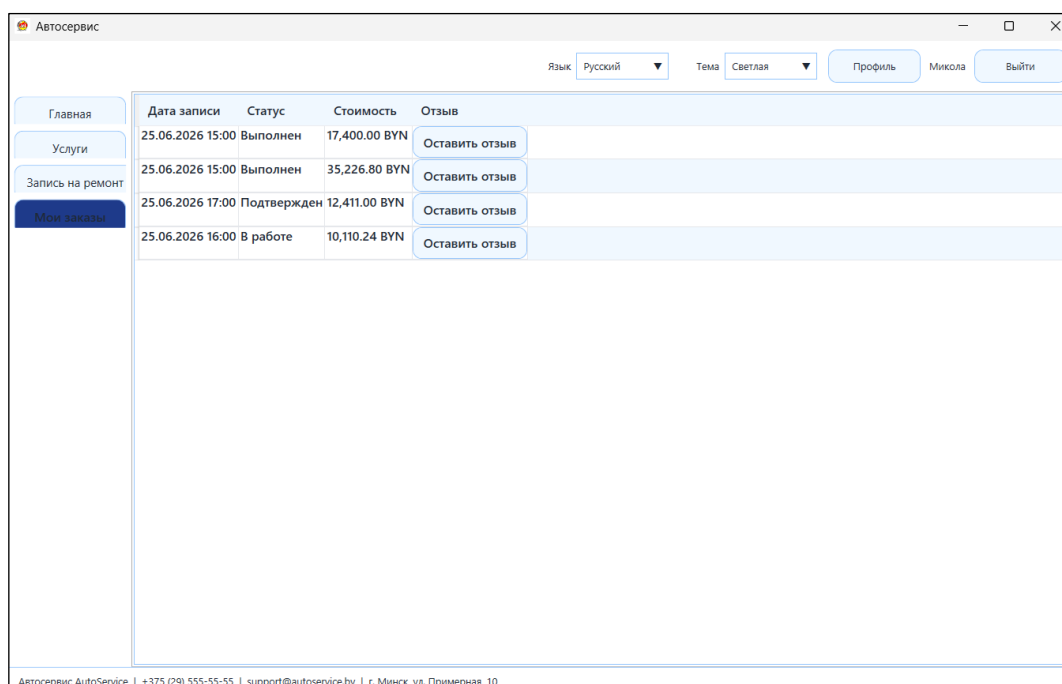


Рисунок 6.5 – Вкладка «Мои заказы»

О смене статуса заказа и других событиях клиент может узнавать из уведомлений в приложении (если предусмотрено вашей сборкой) и, при настройке почты, из e-mail-уведомлений, отправляемых системой при определённых действиях администратора и механика.

Через кнопку «Профиль» в шапке главного окна открывается окно профиля, где можно просмотреть и при необходимости изменить личные данные – рисунок 6.6.

Личный кабинет

ФИО: Микола

Телефон: 80295584578

Язык: Русский ▼

Тема: Светлая ▼

Сохранить

Рисунок 6.6 – Окно профиля пользователя

Выход из аккаунта выполняется кнопкой выхода в правой верхней части главного окна (или сочетанием Ctrl+L).

6.3 Использование приложения в роли администратора

После входа пользователь с ролью «Администратор» видит панель администратора с вкладками слева: «Заказы», «Расписание», «Запчасти», «Услуги» (рисунок 6.7).

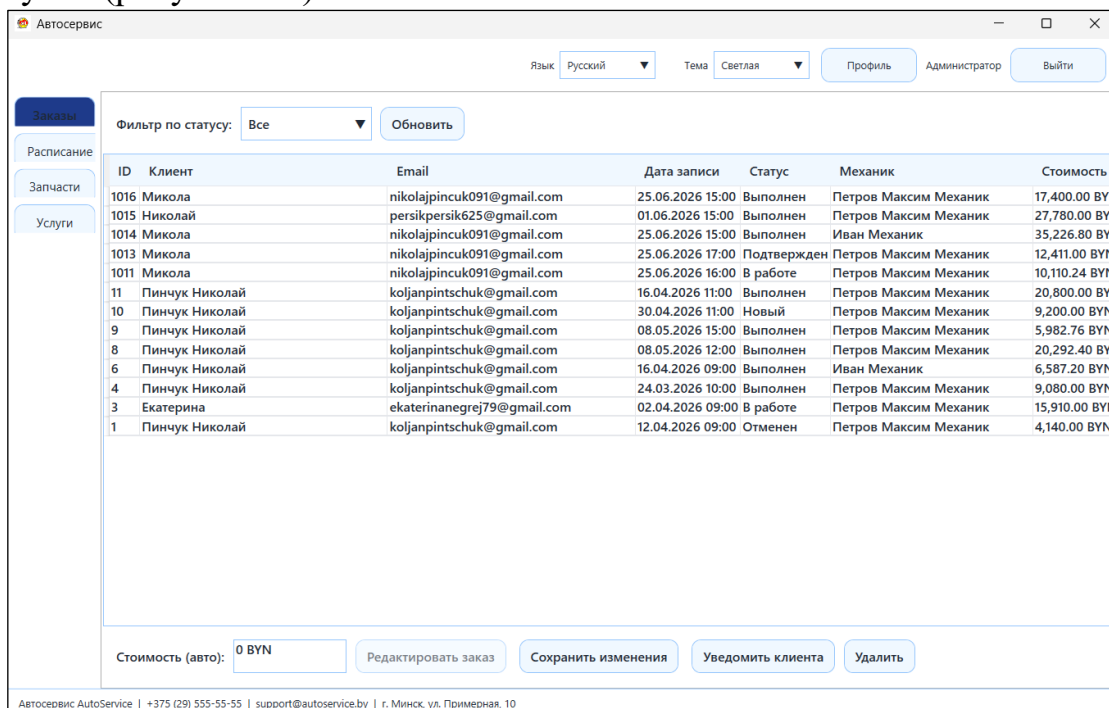


Рисунок 6.7 – Окно заказы приложения (роль «Администратор»)

Вкладка «Заказы». Таблица всех заказов: идентификатор, клиент, e-mail, дата записи, статус, назначенный механик, стоимость. Доступны фильтр по статусу и кнопка обновления списка. При выборе строки заказа внизу отображается пересчитанная стоимость (при необходимости её можно пересчитать и сохранить). Кнопки: редактирование заказа (назначение механика, смена статуса, состав услуг – в диалоговом окне), уведомление клиента, удаление заказа.

Вкладка «Расписание». Отображение записей на выбранный день или всех заказов (флажок режима). Таблица: время записи, клиент, механик, статус, число услуг, стоимость. Для строки доступны действия: настройка услуг по записи (открытие диалога) и отмена записи (перевод в статус отмены с уведомлением клиента). Кнопка оформления записи от имени клиента открывает диалог создания записи (выбор клиента, слота, услуг). Рисунок 6.8.

Автосервис

Язык: Русский | Тема: Светлая | Профиль | Администратор | Выйти

Заказы | ☒ Показать все записи | Дата: | Записать клиента

ID	Дата и время	Клиент	Механик	Статус	Услуги	Стоимость	Действие
4	24.03.2026 10:00	Пинчук Николай	Петров Максим Механик	Выполнен	3	9,080.00 BYN	Услуги Отменить запись
3	02.04.2026 09:00	Екатерина	Петров Максим Механик	В работе	4	15,910.00 BYN	Услуги Отменить запись
1	12.04.2026 09:00	Пинчук Николай	Петров Максим Механик	Отменен	3	4,140.00 BYN	Услуги Отменить запись
6	16.04.2026 09:00	Пинчук Николай	Иван Механик	Выполнен	2	6,587.20 BYN	Услуги Отменить запись
11	16.04.2026 11:00	Пинчук Николай	Петров Максим Механик	Выполнен	3	20,800.00 BYN	Услуги Отменить запись
10	30.04.2026 11:00	Пинчук Николай	Петров Максим Механик	Новый	4	9,200.00 BYN	Услуги Отменить запись
8	08.05.2026 12:00	Пинчук Николай	Петров Максим Механик	Выполнен	3	20,292.40 BYN	Услуги Отменить запись
9	08.05.2026 16:00	Пинчук Николай	Петров Максим Механик	Выполнен	3	5,982.76 BYN	Услуги Отменить запись
1015	01.06.2026 15:00	Николай	Петров Максим Механик	Выполнен	3	27,780.00 BYN	Услуги Отменить запись
1014	25.06.2026 15:00	Микола	Иван Механик	Выполнен	6	35,226.80 BYN	Услуги Отменить запись
1016	25.06.2026 15:00	Микола	Петров Максим Механик	Выполнен	1	17,400.00 BYN	Услуги Отменить запись
1011	25.06.2026 16:00	Микола	Петров Максим Механик	В работе	4	10,110.24 BYN	Услуги Отменить запись
1013	25.06.2026 17:00	Микола	Петров Максим Механик	Подтвержден	5	12,411.00 BYN	Услуги Отменить запись

Автосервис AutoService | +375 (29) 555-55-55 | support@autoservice.by | г. Минск, ул. Примерная, 10

Рисунок 6.8 – Вкладка «Расписание»

Вкладка «Запчасти». Поиск по названию или артикулу, добавление, редактирование и удаление позиций склада, таблица с количеством и ценой. В нижней части – список заявок механиков в статусе «Ожидает» с кнопками «Утвердить» и «Отклонить»; при утверждении учитывается наличие на складе, остатки и статусы заказов обновляются по правилам программы. Рисунок 6.9.

Автосервис

Язык: Русский | Тема: Светлая | Профиль | Администратор | Выйти

Заказы | | Поиск | Добавить запчасть

Название	Артикул	Количество	Цена	Действие
Аккумулятор 60Ah	BAT-60	0	4,200.00 BYN	Редактировать Удалить
Сцепление комплект	CLUTCH-KIT	3	1,250.00 BYN	Редактировать Удалить
Амортизатор задний	SHOCK-R	5	4,800.00 BYN	Редактировать Удалить
Подшипник ступицы 030	ABS-0031	5	920.00 BYN	Редактировать Удалить
Подшипник ступицы 060	ABS-0061	5	1,000.00 BYN	Редактировать Удалить
Подшипник ступицы 090	ABS-0091	5	2,660.00 BYN	Редактировать Удалить
Подшипник ступицы 1020	ABS-1021	5	5,630.00 BYN	Редактировать Удалить
Подшипник ступицы 1050	ABS-1051	5	6,500.00 BYN	Редактировать Удалить
Подшипник ступицы 1080	ABS-1081	5	7,370.00 BYN	Редактировать Удалить
Подшипник ступицы 1110	ABS-1111	5	8,240.00 BYN	Редактировать Удалить
Подшипник ступицы 1140	ABS-1141	5	9,110.00 BYN	Редактировать Удалить

Запросы от механиков:

Заказ	Запчасть	Кол-во	Статус	Действие
-------	----------	--------	--------	----------

Автосервис AutoService | +375 (29) 555-55-55 | support@autoservice.by | г. Минск, ул. Примерная, 10

Рисунок 6.9 – Вкладка «Запчасти» и заявки механиков

Вкладка «Услуги». Ведение справочника услуг: поиск, добавление, редактирование, удаление. Удаление недоступно для услуг, уже включённых в заказы (выводится предупреждение). Рисунок 6.10.

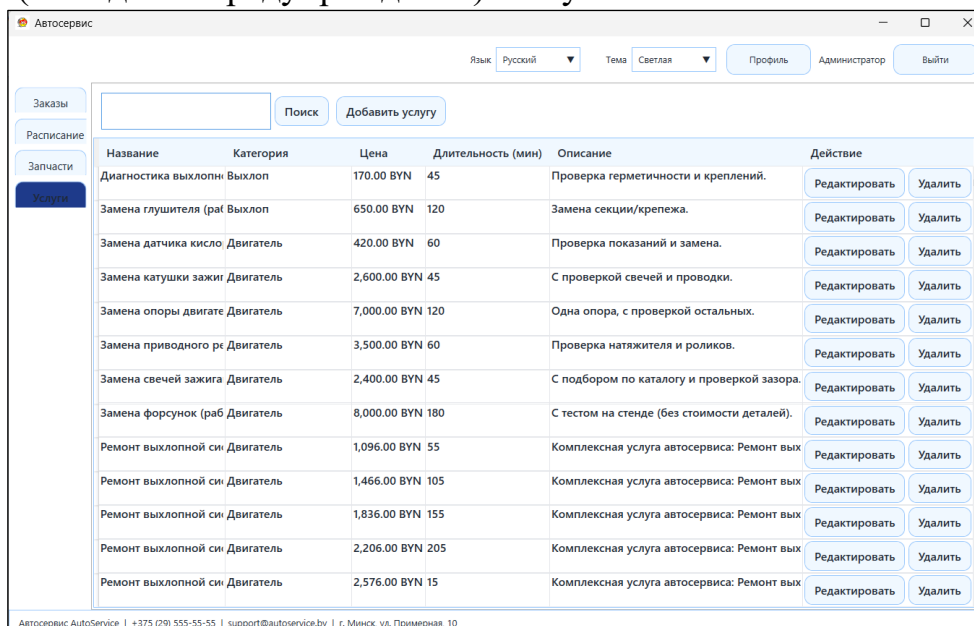


Рисунок 6.10 – Вкладка «Услуги» администратора

Профиль, смена языка и темы, выход из системы – так же, как у клиента (раздел 6.2).

6.4 Использование приложения в роли механика

Пользователь с ролью «Механик» получает панель механика (рисунок 6.11): список назначенных заказов и справочник доступных на складе запчастей с положительным остатком.

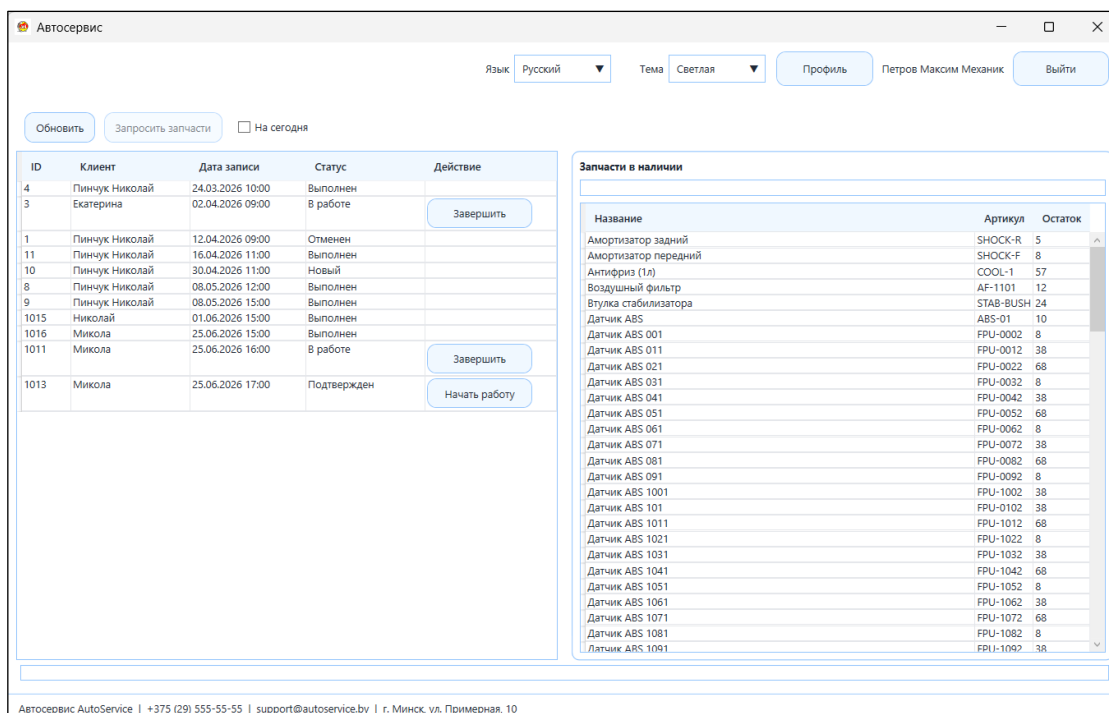


Рисунок 6.11 – Главное окно приложения (роль «Механик»)

В верхней части панели: кнопка «Обновить», кнопка «Запросить запчасти» (активна при выбранном заказе, для которого допустим запрос), флажок «На сегодня» – ограничивает список заказов текущей датой записи.

Таблица заказов содержит идентификатор, клиента, дату записи, статус. В зависимости от статуса отображаются кнопки:

«Начать работу» – для заказа в статусе «Подтвержден»;

«Завершить» – для заказа в статусе «В работе» (перевод в «Выполнен», пересчёт стоимости, уведомление клиенту и при необходимости формирование квитанции по логике приложения).

Поиск по заданиям: в нижней части панели расположено поле ввода; фильтрация выполняется по ФИО клиента или названию услуги в составе заказа.

Справочник запчастей справа: заголовок блока, поле поиска по названию или артикулу, таблица с остатками – для ориентира при работе и при составлении заявки.

Запрос дополнительных запчастей: выберите заказ в таблице, нажмите «Запросить запчасти»; в открывшемся диалоге укажите детали заявки. После отправки администратор получает уведомление; заявка отображается на вкладке «Запчасти» у администратора.

Обновление данных – кнопка «Обновить» или F5 в главном окне. Профиль, язык, тема и выход – как в разделе 6.2.

6.5 Заключение

В разделе описан порядок авторизации и регистрации, работа клиента (просмотр услуг и цен, запись на ремонт, отслеживание заказов и отзывы), администратора (заказы, расписание, склад запчастей, заявки механиков, справочник услуг, уведомления клиентам) и механика (задания, смена статуса выполнения, поиск, запрос запчастей). Данное руководство поможет новым пользователям быстрее освоить возможности приложения. В разделе регистрации подробно разобраны требования к паролю (минимальная длина, наличие цифр и букв) и примеры сообщений об ошибках при вводе некорректных данных.

Для клиента приведены пошаговые сценарии: выбор услуги из каталога с фильтром по цене и длительности, а также инструкция по отмене записи с автоматическим возвратом свободного слота в расписание.

В части администратора детально показано, как формируются отчёты по загруженности механиков и как обрабатываются заявки на недостающие запчасти с последующим оповещением склада.

Описание работы механика сопровождается скриншотами интерфейса с пояснением.

Заключение

Разработанное программное средство "Автосервис" представляет собой значительный шаг вперед в сфере управления продажами и учетом автозапчастей. Данное приложение обеспечивает удобство, эффективность и простоту в организации процессов как для продавцов, так и для клиентов.

Целью разработки являлось создание инновационного программного решения, удовлетворяющего современные потребности пользователей в сфере автобизнеса. Основные принципы, заложенные в разработку, включают разнообразие функционала, а также интуитивно понятный интерфейс для легкого освоения пользователями. Это приложение может стать удобным инструментом в современной бизнес-среде, способствуя повышению эффективности работы профессионалов.

Одним из главных преимуществ программного средства является его простота использования. Дружественный пользовательский интерфейс и интуитивно понятные функции делают работу с приложением комфортной и без лишних сложностей. Пользователи смогут быстро освоиться с программой и использовать ее по назначению без необходимости дополнительного обучения.

В процессе реализации программного средства были достигнуты вышеуказанные требования и задачи. Применение паттерна MVVM обеспечило эффективное управление данными и логикой приложения, улучшенную архитектуру и обеспечило высокую отзывчивость и удобство использования пользовательского интерфейса.

Работа над проектом также позволила получить ценный опыт взаимодействия со сторонними библиотеками. Изучение и внедрение этих библиотек в проект позволило расширить его функциональность и улучшить его производительность. Разработанное программное средство "Автосервис" является функциональным, удобным и привлекательным для пользователей. Внедрение ролевой модели доступа (клиент, администратор, механик) позволило точно разграничить функциональные возможности и обеспечить информационную безопасность на всех уровнях приложения.

Благодаря использованию асинхронных операций интерфейс остаётся отзывчивым даже при выполнении ресурсоёмких запросов, таких как формирование отчётов или поиск по каталогу запчастей.

Система уведомлений, интегрированная с мессенджерами и электронной почтой, обеспечивает своевременное информирование всех участников процесса об изменении статуса заказа или поступлении запрошенных деталей.

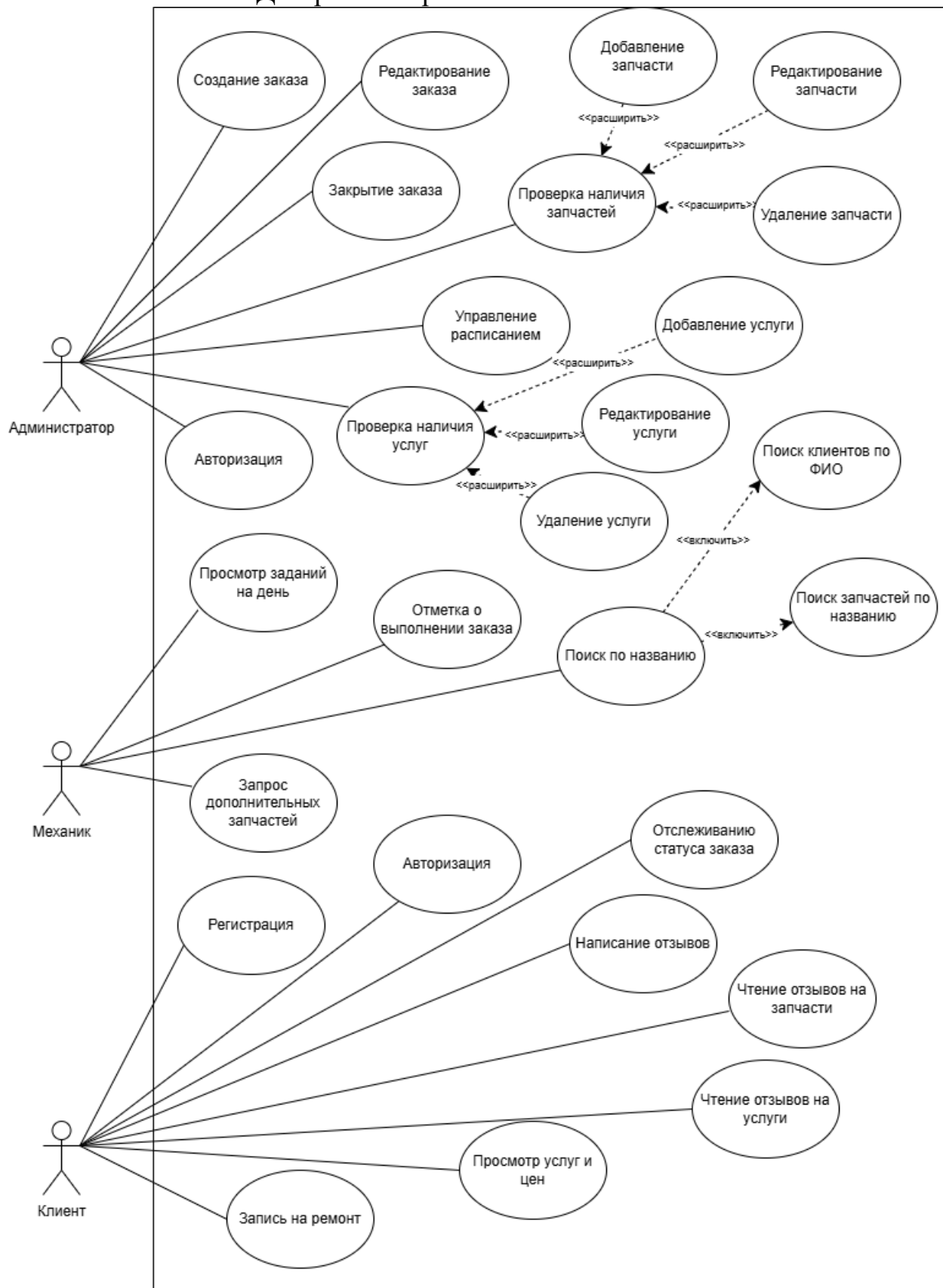
Модуль аналитики, доступный администратору, позволяет отслеживать ключевые метрики: загрузку механиков, средний чек, оборачиваемость заказов и наиболее востребованные услуги.

Список использованных источников

- 1 Онлайн-ресурс «fitauto.ru» [Электронный ресурс] – Режим доступа: <https://free.fitauto.ru/> – Дата доступа: 10.04.2026
- 2 Онлайн-ресурс «willgood.ru» [Электронный ресурс] – Режим доступа: <https://willgood.ru/autoservice/> – Дата доступа: 10.04.2026
- 3 Онлайн-ресурс «arserv.by» [Электронный ресурс] – Режим доступа: <https://arserv.by/> – Дата доступа: 10.04.2026
- 4 Microsoft Visual Studio [Электронный ресурс] – Режим доступа: https://ru.wikipedia.org/wiki/Microsoft_Visual_Studio – Дата доступа: 15.04.2026
- 5 Работа с Entity Framework [Электронный ресурс] – Режим доступа: <https://learn.microsoft.com/ru-ru/ef/> – Дата доступа: 10.04.2026
- 6 Руководство по WPF [Электронный ресурс] – Режим доступа: <https://metanit.com/sharp/wpf/> – Дата доступа: 14.04.2026
- 7 Руководство по XAML [Электронный ресурс] – Режим доступа: <https://www.tutorialspoint.com/xaml/index.htm> – Дата доступа: 16.04.2026
- 8 Документация MSSQL [Электронный ресурс] – Режим доступа: <https://www.postgresql.org/docs/> – Дата доступа: 22.04.2026

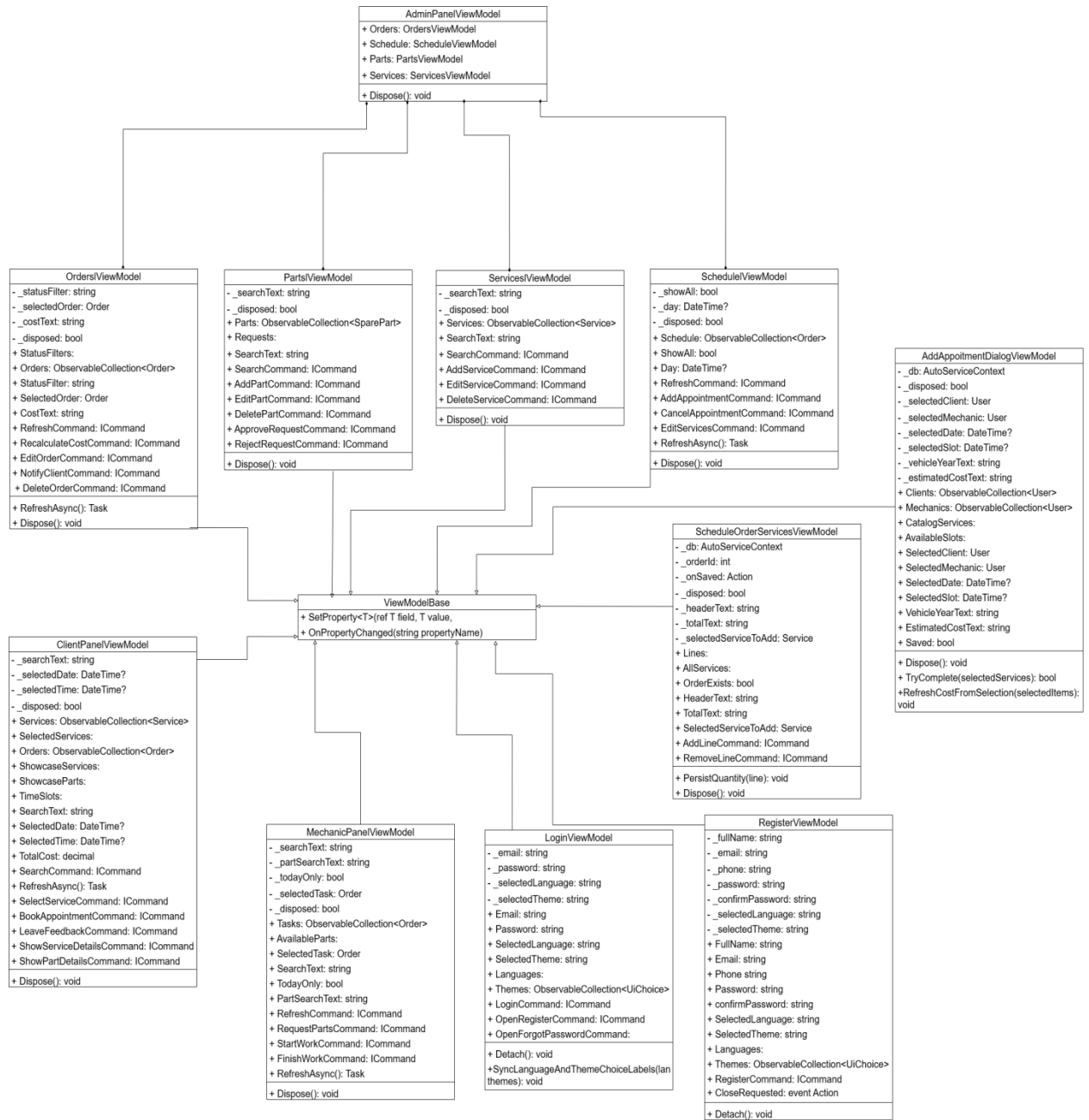
ПРИЛОЖЕНИЕ А

Диаграмма вариантов использования



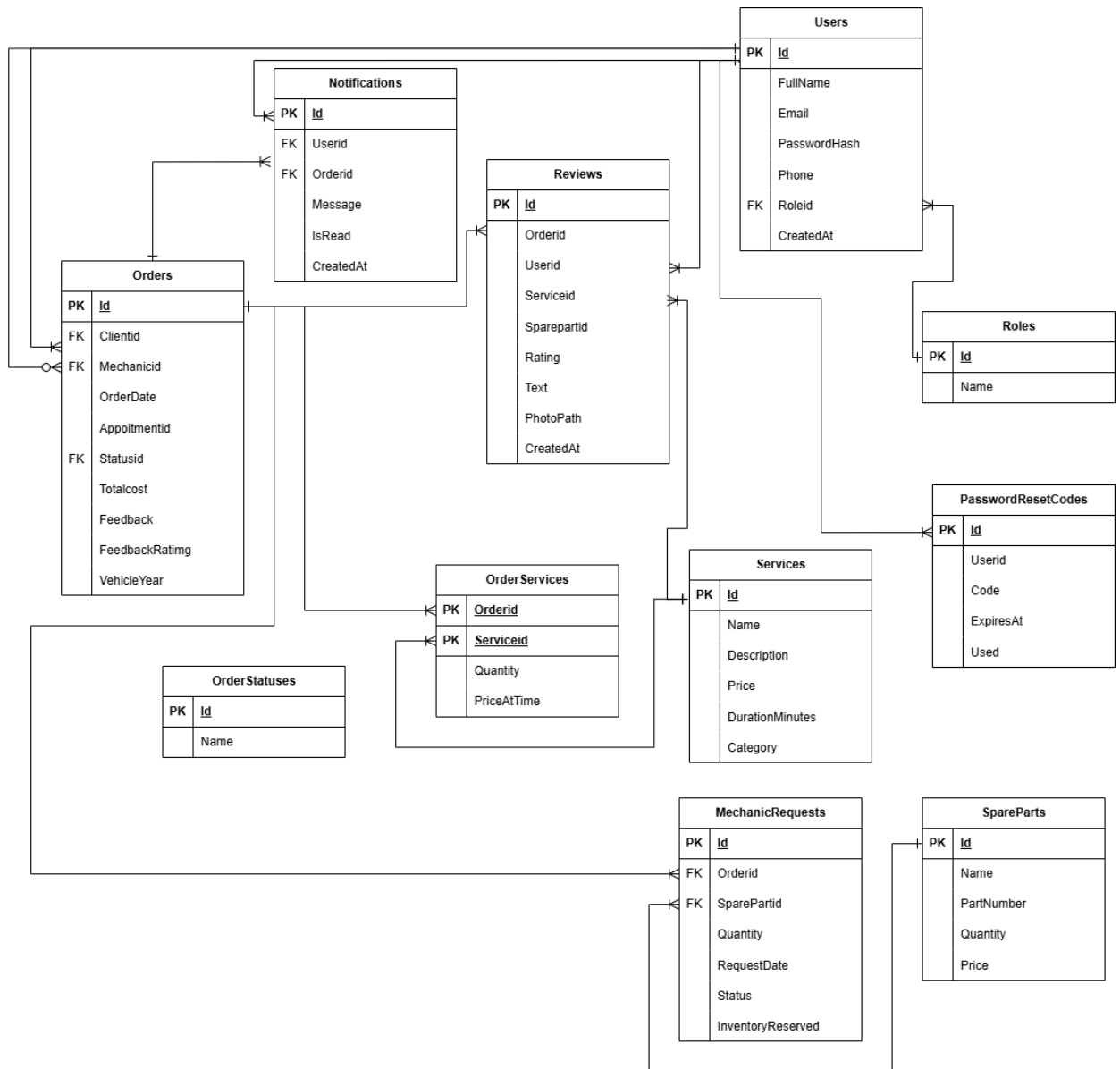
ПРИЛОЖЕНИЕ Б

Диаграмма классов ViewModel



Приложение В

Логическая схема базы данных



Приложение Г

Код представления окна авторизации

```
<Window x:Class="AutoServiceApp.Views.LoginWindow"

xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:vm="clr-namespace:AutoServiceApp.ViewModels"
    xmlns:mvvm="clr-namespace:AutoServiceApp.Mvvm"
    Title="Авторизация" Height="400" Width="480"
WindowStartupLocation="CenterScreen"
    Icon="pack://application:,,,/Assets/app.ico">
    <Grid Margin="20">
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="*" />
        </Grid.RowDefinitions>
        <Grid.ColumnDefinitions>
            <ColumnDefinition Width="Auto"/>
            <ColumnDefinition Width="*" />
            <ColumnDefinition Width="Auto"/>
        </Grid.ColumnDefinitions>

        <StackPanel Grid.Row="0" Grid.Column="1"
Grid.ColumnSpan="2" Orientation="Horizontal"
HorizontalAlignment="Right" Margin="5">
            <TextBlock x:Name="txtLanguageCaption" Width="62"
TextAlignment="Right" VerticalAlignment="Center"
Margin="0,0,6,0"/>
            <ComboBox Width="110" Height="32" Margin="0,0,8,0"
ItemsSource="{Binding Languages}"
DisplayMemberPath="Title"
SelectedValuePath="Id"
SelectedValue="{Binding SelectedLanguage,
Mode=TwoWay, UpdateSourceTrigger=PropertyChanged}">
            </ComboBox>
            <TextBlock x:Name="txtThemeCaption" Width="52"
TextAlignment="Right" VerticalAlignment="Center"
Margin="0,0,6,0"/>
            <ComboBox Width="110" Height="32"
ItemsSource="{Binding Themes}"
DisplayMemberPath="Title"
SelectedValuePath="Id"
SelectedValue="{Binding SelectedTheme,
Mode=TwoWay, UpdateSourceTrigger=PropertyChanged}">
            </ComboBox>
    </Grid>
</Window>
```

```

        </StackPanel>

        <Label x:Name="lblEmail" Grid.Row="1" Grid.Column="0"
Margin="5"/>
        <TextBox Grid.Row="1" Grid.Column="1"
Grid.ColumnSpan="2" Margin="5"
Text="{Binding Email, Mode=TwoWay,
UpdateSourceTrigger=PropertyChanged}"/>

        <Label x:Name="lblPassword" Grid.Row="2" Grid.Column="0"
Margin="5"/>
        <Border Grid.Row="2"
Grid.Column="1"
Grid.ColumnSpan="2"
Margin="5"
BorderBrush="{DynamicResource BorderBrush}"
BorderThickness="1"
Background="{DynamicResource
ControlBackgroundBrush}"
CornerRadius="10">
            <Grid>
                <Grid.ColumnDefinitions>
                    <ColumnDefinition Width="*" />
                    <ColumnDefinition Width="Auto" />
                </Grid.ColumnDefinitions>

                <PasswordBox x:Name="txtPassword"
Grid.Column="0"
Margin="10,5,4,5"
Background="Transparent"
BorderThickness="0"

mvvm:PasswordBoxAssistant.BindPassword="True"

mvvm:PasswordBoxAssistant.BoundPassword="{Binding Password,
Mode=TwoWay, UpdateSourceTrigger=PropertyChanged}"
Visibility="Visible"/>
                <TextBox x:Name="txtPasswordVisible"
Grid.Column="0"
Margin="10,5,4,5"
Background="Transparent"
BorderThickness="0"
Text="{Binding Password, Mode=TwoWay,
UpdateSourceTrigger=PropertyChanged}"
Visibility="Collapsed"/>
                <Button x:Name="btnTogglePassword"
Grid.Column="1"
Content="□"
Width="34"
Margin="0,2,2,2"
Padding="0"
Click="TogglePasswordVisibility"/>
            </Grid>

```

```
</Border>

    <Button x:Name="btnLogin" Grid.Row="3" Grid.Column="1"
Margin="5" Command="{Binding LoginCommand}"/>
    <Button x:Name="btnRegister" Grid.Row="4"
Grid.Column="1" Margin="5" Command="{Binding
OpenRegisterCommand}"/>
    <Button x:Name="btnForgotPassword" Grid.Row="5"
Grid.Column="1" Margin="5" HorizontalAlignment="Left"
Command="{Binding OpenForgotPasswordCommand}"
Background="Transparent" BorderThickness="0" Cursor="Hand"/>
    </Grid>
</Window>
```

ПРИЛОЖЕНИЕ Г

Код модели представления окна авторизации

```
using System;
using System.Collections.ObjectModel;
using System.ComponentModel;
using System.Runtime.CompilerServices;
using System.Threading.Tasks;
using System.Windows;
using System.Windows.Input;
using AutoServiceApp.Mvvm;
using AutoServiceApp.Services;
using AutoServiceApp.Services.Abstractions;

namespace AutoServiceApp.ViewModels
{
    public sealed class LoginViewModel : ViewModelBase
    {
        private readonly IAuthService _auth;
        private readonly IAppSession _session;
        private readonly INavigationService _nav;
        private readonly IDialogService _dialogs;

        private string _email;
        private string _password;
        private string _selectedLanguage;
        private string _selectedTheme;

        public LoginViewModel(IAuthService auth, IAppSession session, INavigationService nav, IDialogService dialogs)
        {
            _auth = auth ?? throw new ArgumentNullException(nameof(auth));
            _session = session ?? throw new ArgumentNullException(nameof(session));
            _nav = nav ?? throw new ArgumentNullException(nameof(nav));
            _dialogs = dialogs ?? throw new ArgumentNullException(nameof(dialogs));

            Languages = new ObservableCollection<UiChoice>();
            Themes = new ObservableCollection<UiChoice>();

            _selectedLanguage = LocalizationService.CurrentLanguage ?? "ru";
            _selectedTheme = ThemeManager.CurrentTheme ?? "light";

            LoginCommand = new AsyncRelayCommand(LoginAsync);
        }
    }
}
```

```

        OpenRegisterCommand = new
RelayCommand(OpenRegister);
        OpenForgotPasswordCommand = new
RelayCommand(OpenForgotPassword);

        LocalizationService.LanguageChanged +=
OnLanguageChanged;
        RefreshSelectorLabels();
    }

    public void Detach()
    {
        LocalizationService.LanguageChanged -=
OnLanguageChanged;
    }

    private void OnLanguageChanged()
    {
        var app = Application.Current;
        if (app?.Dispatcher != null &&
!app.Dispatcher.CheckAccess())
        {
            app.Dispatcher.Invoke(RefreshSelectorLabels);
        }
        else
        {
            RefreshSelectorLabels();
        }
    }

    /// <remarks>Не вызывать сеттеры
SelectedLanguage/SelectedTheme – иначе повторный SetLanguage и
переполнение стека.
    /// Подписи обновляем на месте без Clear(), иначе
открытый ComboBox в WPF дублирует пункты.</remarks>
    private void RefreshSelectorLabels()
    {
        var lang =
string.IsNullOrEmpty(_selectedLanguage) ? "ru" :
_selectedLanguage;
        var theme =
string.IsNullOrEmpty(_selectedTheme) ? "light" :
_selectedTheme;

        SyncLanguageAndThemeChoiceLabels(Languages, Themes);

        _selectedLanguage = lang;
        _selectedTheme = theme;
        OnPropertyChanged(nameof(SelectedLanguage));
        OnPropertyChanged(nameof(SelectedTheme));
    }

```

```

        /// <summary>Обновляет подписи языка/темы без Clear
коллекции (избегает дубликатов в ComboBox).</summary>
        public static void SyncLanguageAndThemeChoiceLabels(
            ObservableCollection<UiChoice> languages,
            ObservableCollection<UiChoice> themes)
        {
            EnsureChoices(languages, new[] { ("ru",
"LanguageRu"), ("en", "LanguageEn") });
            EnsureChoices(themes, new[] { ("light",
"ThemeLight"), ("dark", "ThemeDark"), ("pink", "ThemePink"),
("gray", "ThemeGray") });

            foreach (var item in languages)
            {
                item.Title =
LocalizationService.Get(item.TitleLocKey);
            }

            foreach (var item in themes)
            {
                item.Title =
LocalizationService.Get(item.TitleLocKey);
            }
        }

        private static void
EnsureChoices(ObservableCollection<UiChoice> list, (string id,
string locKey)[] spec)
        {
            if (list.Count != spec.Length)
            {
                list.Clear();
                foreach (var (id, locKey) in spec)
                {
                    list.Add(new UiChoice(id,
LocalizationService.Get(locKey), locKey));
                }

                return;
            }

            for (var i = 0; i < spec.Length; i++)
            {
                if (list[i].Id != spec[i].id)
                {
                    list.Clear();
                    foreach (var (id, locKey) in spec)
                    {
                        list.Add(new UiChoice(id,
LocalizationService.Get(locKey), locKey));
                    }

                    return;
                }
            }
        }
    }
}

```

```

        }

        list[i].TitleLocKey = spec[i].locKey;
    }
}

public ObservableCollection<UiChoice> Languages { get; }
public ObservableCollection<UiChoice> Themes { get; }

public string Email
{
    get => _email;
    set => SetProperty(ref _email, value);
}

public string Password
{
    get => _password;
    set => SetProperty(ref _password, value);
}

public string SelectedLanguage
{
    get => _selectedLanguage;
    set
    {
        if (SetProperty(ref _selectedLanguage, value))
        {
            LocalizationService.SetLanguage(value);
        }
    }
}

public string SelectedTheme
{
    get => _selectedTheme;
    set
    {
        if (SetProperty(ref _selectedTheme, value))
        {
            ThemeManager.ApplyTheme(value);
        }
    }
}

public ICommand LoginCommand { get; }
public ICommand OpenRegisterCommand { get; }
public ICommand OpenForgotPasswordCommand { get; }

private async Task LoginAsync()
{
    var email = (Email ?? string.Empty).Trim();
    var password = Password ?? string.Empty;

```

```

        if (string.IsNullOrEmpty(email) ||
string.IsNullOrEmpty(password))
        {

            _dialogs.ShowWarning(LocalizationService.Get("FillRequired"),
LocalizationService.Get("Error"));
            return;
        }

        var user = await _auth.LoginAsync(email, password);
        if (user == null)
        {

            _dialogs.ShowError(LocalizationService.Get("UserNotFound"),
LocalizationService.Get("Error"));
            return;
        }

        _session.CurrentUser = user;
        App.CurrentUser = user;
        _nav.OpenMainAndCloseCurrent();
    }

    private void OpenRegister()
    {
        _nav.OpenRegisterDialog(App.Current?.MainWindow);
    }

    private void OpenForgotPassword()
    {
        _nav.OpenForgotPasswordDialog(App.Current?.MainWindow);
    }

    public sealed class UiChoice : INotifyPropertyChanged
    {
        private string _title;
        private string _titleLocKey;

        public UiChoice(string id, string title, string
titleLocKey)
        {
            Id = id;
            _title = title;
            _titleLocKey = titleLocKey;
        }

        public string Id { get; }

        /// <summary>Ключ строки в LocalizationService
(LanguageRu, ThemeLight, ...).</summary>
        public string TitleLocKey
        {

```

```

        get => _titleLocKey;
        set
        {
            if (_titleLocKey == value) return;
            _titleLocKey = value;
            OnPropertyChanged();
        }
    }

    public string Title
    {
        get => _title;
        set
        {
            if (_title == value) return;
            _title = value;
            OnPropertyChanged();
        }
    }

    public event PropertyChangedEventHandler
PropertyChanged;

    private void OnPropertyChanged([CallerMemberName]
string name = null)
    {
        PropertyChanged?.Invoke(this, new
PropertyChangedEventArgs(name));
    }

    public override string ToString() => Title ?? Id ??
string.Empty;
    }
}

```

ПРИЛОЖЕНИЕ Д

Код класса AutoServiceContext

```
using AutoServiceApp.Models;
using System.Data.Entity;

namespace AutoServiceApp.Context
{
    public class AutoServiceContext : DbContext
    {
        public AutoServiceContext() :
base("name=AutoServiceContext")
        {
            Database.SetInitializer<AutoServiceContext>(null);
        }

        public DbSet<Role> Roles { get; set; }
        public DbSet<User> Users { get; set; }
        public DbSet<Service> Services { get; set; }
        public DbSet<Order> Orders { get; set; }
        public DbSet<OrderStatus> OrderStatuses { get; set; }
        public DbSet<OrderService> OrderServices { get; set; }
        public DbSet<SparePart> SpareParts { get; set; }
        public DbSet<MechanicRequest> MechanicRequests { get;
set; }
        public DbSet<Notification> Notifications { get; set; }
        public DbSet<PasswordResetCode> PasswordResetCodes {
get; set; }
        public DbSet<Review> Reviews { get; set; }

        protected override void OnModelCreating(DbModelBuilder
modelBuilder)
        {
            modelBuilder.Entity<OrderService>()
                .HasKey(os => new { os.OrderId, os.ServiceId });

            base.OnModelCreating(modelBuilder);
        }
    }
}
```

ПРИЛОЖЕНИЕ Ж

Код модели представления окна регистрации

```
using System;
using System.Collections.ObjectModel;
using System.Text.RegularExpressions;
using System.Threading.Tasks;
using System.Windows;
using System.Windows.Input;
using AutoServiceApp.Mvvm;
using AutoServiceApp.Services;
using AutoServiceApp.Services.Abstractions;

namespace AutoServiceApp.ViewModels
{
    public sealed class RegisterViewModel : ViewModelBase
    {
        private readonly IAuthService _auth;
        private readonly IDialogService _dialogs;

        private string _fullName;
        private string _email;
        private string _phone;
        private string _password;
        private string _confirmPassword;
        private string _selectedLanguage;
        private string _selectedTheme;

        public RegisterViewModel(IAuthService auth,
IDialogService dialogs)
        {
            _auth = auth ?? throw new
ArgumentNullException(nameof(auth));
            _dialogs = dialogs ?? throw new
ArgumentNullException(nameof(dialogs));

            Languages = new
ObservableCollection<LoginViewModel.UiChoice>();
            Themes = new
ObservableCollection<LoginViewModel.UiChoice>();

            _selectedLanguage =
LocalizationService.CurrentLanguage ?? "ru";
            _selectedTheme = ThemeManager.CurrentTheme ??
"light";

            LocalizationService.LanguageChanged +=
OnLanguageChanged;
            RefreshSelectorLabels();

            RegisterCommand = new
```

```

AsyncRelayCommand(RegisterAsync);
    }

    public void Detach()
    {
        LocalizationService.LanguageChanged -=
OnLanguageChanged;
    }

    private void OnLanguageChanged()
    {
        var app = Application.Current;
        if (app?.Dispatcher != null &&
!app.Dispatcher.CheckAccess())
        {
            app.Dispatcher.Invoke(RefreshSelectorLabels);
        }
        else
        {
            RefreshSelectorLabels();
        }
    }

    /// <remarks>Не вызывать сеттеры
SelectedLanguage/SelectedTheme — иначе повторный SetLanguage и
переполнение стека.</remarks>
    private void RefreshSelectorLabels()
    {
        var lang =
string.IsNullOrEmpty(_selectedLanguage) ? "ru" :
_selectedLanguage;
        var theme =
string.IsNullOrEmpty(_selectedTheme) ? "light" :
_selectedTheme;

LoginViewModel.SyncLanguageAndThemeChoiceLabels(Languages,
Themes);

        _selectedLanguage = lang;
        _selectedTheme = theme;
        OnPropertyChanged(nameof(SelectedLanguage));
        OnPropertyChanged(nameof(SelectedTheme));
    }

    public ObservableCollection<LoginViewModel.UiChoice>
Languages { get; }
    public ObservableCollection<LoginViewModel.UiChoice>
Themes { get; }

    public string FullName
    {
        get => _fullName;
    }

```

```

        set => SetProperty(ref _fullName, value);
    }

    public string Email
    {
        get => _email;
        set => SetProperty(ref _email, value);
    }

    public string Phone
    {
        get => _phone;
        set => SetProperty(ref _phone, value);
    }

    public string Password
    {
        get => _password;
        set => SetProperty(ref _password, value);
    }

    public string ConfirmPassword
    {
        get => _confirmPassword;
        set => SetProperty(ref _confirmPassword, value);
    }

    public string SelectedLanguage
    {
        get => _selectedLanguage;
        set
        {
            if (SetProperty(ref _selectedLanguage, value))
            {
                LocalizationService.SetLanguage(value);
            }
        }
    }

    public string SelectedTheme
    {
        get => _selectedTheme;
        set
        {
            if (SetProperty(ref _selectedTheme, value))
            {
                ThemeManager.ApplyTheme(value);
            }
        }
    }

    public ICommand RegisterCommand { get; }

```

```

private async Task RegisterAsync()
{
    var name = (FullName ?? string.Empty).Trim();
    var email = (Email ?? string.Empty).Trim();
    var phone = (Phone ?? string.Empty).Trim();
    var password = Password ?? string.Empty;
    var confirm = ConfirmPassword ?? string.Empty;

    if (string.IsNullOrEmpty(name) ||
string.IsNullOrEmpty(email) ||
string.IsNullOrEmpty(password))
    {
        _dialogs.ShowWarning(LocalizationService.Get("FillRequired"),
LocalizationService.Get("Error"));
        return;
    }

    if (!IsValidEmail(email))
    {
        _dialogs.ShowWarning(LocalizationService.Get("InvalidEmail"),
LocalizationService.Get("Error"));
        return;
    }

    if (!string.IsNullOrEmpty(phone) &&
!IsValidPhone(phone))
    {
        _dialogs.ShowWarning(LocalizationService.Get("InvalidPhone"),
LocalizationService.Get("Error"));
        return;
    }

    if (!string.Equals(password, confirm,
StringComparison.Ordinal))
    {
        _dialogs.ShowWarning(LocalizationService.Get("PasswordMismatch"),
, LocalizationService.Get("Error"));
        return;
    }

    var (ok, errorKey) = await
_auth.RegisterClientAsync(name, email, phone, password);
    if (!ok)
    {
        _dialogs.ShowError(LocalizationService.Get(errorKey ?? "Error"),
LocalizationService.Get("Error"));
        return;
    }
}

```

```

        _dialogs.ShowInfo(LocalizationService.Get("RegistrationDone"),
LocalizationService.Get("Success"));
        CloseRequested?.Invoke();
    }

    public event Action CloseRequested;

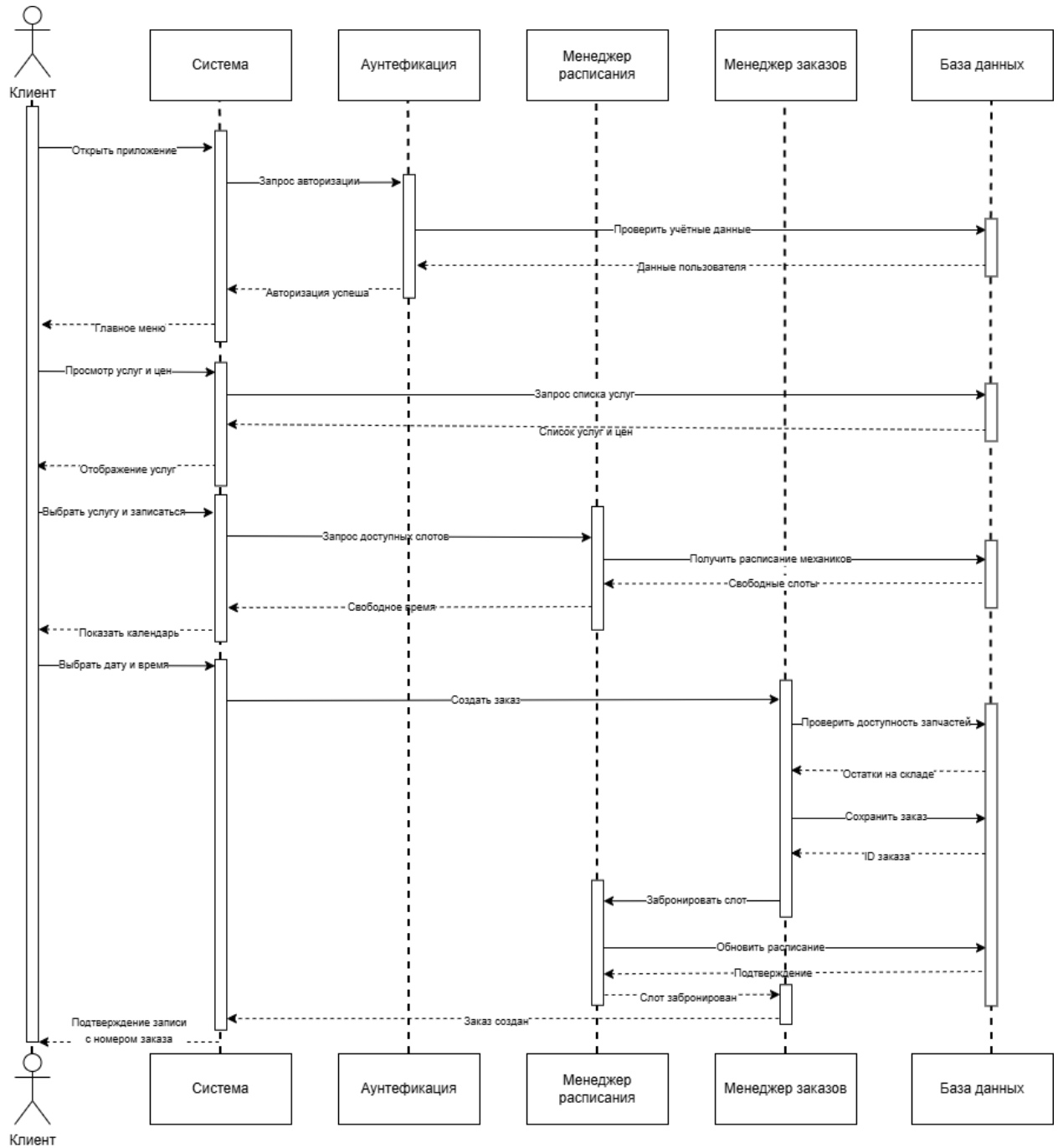
    private static bool IsValidEmail(string email)
    {
        try
        {
            var addr = new
System.Net.Mail.MailAddress(email);
            return addr.Address == email;
        }
        catch
        {
            return false;
        }
    }

    private static bool IsValidPhone(string phone)
    {
        var cleaned = Regex.Replace(phone, @"^\d+", "");
        if (cleaned.StartsWith("+375") && cleaned.Length ==
13) return true;
        if (cleaned.StartsWith("80") && cleaned.Length ==
11) return true;
        if (cleaned.Length >= 9 && cleaned.Length <= 13 &&
Regex.IsMatch(cleaned, @"^\d+$")) return true;
        return false;
    }
}

```

ПРИЛОЖЕНИЕ 3

Диаграмма последовательности записи клиента



ПРИЛОЖЕНИЕ И

Блок-схема алгоритма записи

